

分类号_____

学校代码 10487

学号 M201272698

密级_____

华中科技大学 硕士学位论文

基于纠删码的光盘库 数据纠错技术研究

学位申请人：张效群

学 科 专 业：计算机系统结构

指 导 教 师：林 安 副教授

答 辩 日 期：2015.5.20

**A Thesis Submitted in Partial Fulfillment of the Requirements
for the Degree for the Master of Engineering**

**Study on Data Recovery Technology
by Erasure Code in Optical Library**

Candidate : Zhang Xiaoqun

Major : Computer Architecture

Supervisor : Assoc. Prof. Lin An

Huazhong University of Science & Technology

Wuhan 430074, P.R.China

May, 2015

摘要

随着大数据时代的到来，数据归档对于企事业单位的作用愈加重要。光盘库因其低廉的价格成为近年来快速发展的海量存储设备。目前，存储系统在容量急剧增长及应用场景多元化的同时，其可靠性也受到越来越多的重视，纠删码作为存储系统纠错的主要方法其重要性日益凸显。

通过分析光盘库作为归档类型存储系统对高纠错能力以及高存储利用率的特殊需求，以 Cauchy-based Reed-Solomon 编码为基础，对光盘库多盘片失效的纠错技术进行了研究和实现。

依据光盘库多盘片失效的实际特点，设计了一种具有高纠错能力的光盘库冗余纠错子系统。该系统由五个模块组成，基本编码模块通过数据光盘文件生成校验光盘文件；基本解码模块在发生盘片失效时，重构失效盘片文件；另外三个模块是编解码过程的公共服务模块。针对光盘编码组规模较大等特点，设计了编解码分段模块，将一个编码过程或解码过程分为多个阶段完成。通过研究原有矩阵调度算法，设计了调度优化算法，从根本上减少了编解码过程异或运算的总数。编解码并发模块采用 Linux 线程池技术实现，利用纠删码在编码条带间的无关性，并发执行编解码运算，有效提高了编解码速度。

最后，通过冗余纠错子系统进行光盘文件分段编码，删除部分光盘文件，再进行分段解码，测试显示该系统在功能上可有效地应对光盘库多盘片失效的情况。测试新型矩阵调度优化算法显示，该算法在性能上可有效减少异或运算总数，其优化能力在编码时约为原有调度算法的 2 倍，在解码时约为 1.7 倍。

关键词：光盘库，数据纠错，纠删码，调度优化

Abstract

With the coming of big data era, data archiving is becoming more and more important for enterprise. Because of its low price, the optical library has developed so quickly in recent years as a kind of mass storage device. With the increasingly complexity of application environment, the reliability of storage system such as optical library was given a severe challenge. As the main method of fault tolerance of storage system, erasure codes are getting more and more attention.

Through the analysis of the specific requirements of optical library which needs high fault tolerance and high space utilization, we studied the fault tolerance technologies for multiple failures of optical library and design the optical library redundancy subsystem with high fault tolerance.

The subsystem is composed of five modules, basic encoding module generates redundancy optical disc from data optical disc. The basic decoding module rebuilds failed optical disc when disc failures occurs. With the codec segmentation module, a encoding or decoding process can be divided into multiple stages. Based on the research for matrix scheduling algorithm, the scheduling optimization algorithm can reduces the total number of exclusive or operations, fundamentally. With the Linux thread pool technology, the encoding and decoding performance has been improve effectively.

The testing show that the scheduling optimization algorithm can effectively reduce the total number of exclusive or operations, and the optimization ability is about 2 times of the original one when encoding, and is about 1.7 times when decoding.

Keywords: Optical Library, Fault Tolerance, Erasure Code, Scheduling Optimization

目 录

摘 要	I
Abstract.....	II
1 绪论	
1.1 研究来源、目的与意义.....	(1)
1.2 纠错技术研究现状	(3)
1.3 本文内容与安排.....	(5)
2 冗余纠错基本理论与技术	
2.1 纠删码的相关概念	(6)
2.2 经典冗余纠错理论	(7)
2.3 近年新型纠删码理论	(12)
2.4 本章小结	(14)
3 冗余纠错系统总体设计	
3.1 典型应用的需求分析	(15)
3.2 冗余纠错系统接口设计.....	(17)
3.3 冗余纠错系统概要设计.....	(19)
3.4 冗余纠错功能模块设计.....	(20)
3.5 本章小结	(25)
4 关键模块和算法的研究及实现	
4.1 基本编码模块实现	(27)
4.2 基本解码模块实现	(30)
4.3 异或调度算法研究和优化	(32)
4.4 线程池并发算法研究和实现	(36)

华中科技大学硕士学位论文

4.5	纠错能力与时空开销分析.....	(38)
4.6	本章小结.....	(39)
5	系统测试	
5.1	测试环境.....	(40)
5.2	系统功能测试.....	(40)
5.3	关键算法优化测试.....	(41)
5.4	系统性能优化测试.....	(46)
5.5	测试分析及结论.....	(50)
6	总结与展望	
6.1	全文总结.....	(51)
6.2	不足与展望.....	(52)
	致 谢	(53)
	参考文献	(54)

1 绪论

1.1 研究来源、目的与意义

近些年来，社交网络、移动互联网尤其是大数据技术等得到飞速发展和广泛应用，全球数据量出现指数式急剧增加。根据相关统计结论显示，2008 年全世界累积的信息数据量达到 0.49ZB（1ZB 等于 1 万亿 GB），2009 年达到 0.8ZB，2010 年达到 1.2ZB，2011 年达到 1.82ZB；根据 IBM 的统计分析指出，整个人类发展过程中产生的所有数据量有近 90%是在过去两年内获得的。全世界数据量正遵循摩尔定律按照每十八个月翻番的规律高速增长，根据该规律，全世界数据量到 2020 年将高达 35ZB 之多，而平均到每人的数量将高达 5200GB 以上；如何有效地存储和利用如此海量的数据将是巨大的挑战和空前的机遇。

对于企业而言，随着信息化、无纸化办公的快速变革，企业数据量每天都在急剧膨胀，数据的类型也各不相同；这些数据是企业赖以生存和发展重要基础，某些重要数据的损坏丢失将给企业带来无可挽回的毁灭性打击，因此需要安全、完整的保存；然而，数据量的爆发式增加对存储空间的需求越来越大，而数据存储管理的难度和复杂度也迅速增加。以医疗行业为例分析，随着医疗业信息化快速推进，尤其是电子健康记录、大型医疗影像设备等应用日渐广泛，根据 IDC 相关研究报告指出医疗行业的数据量正以 48%的增速爆炸式增加；而如果对相关数据进行标记并进一步分析研究，是可以利用大数据技术对 57%数据量加以利用的。电子商务领域巨头阿里巴巴同样有着海量的数据，根据阿里数据平台事业部披露的信息，到 2014 年 3 月为止，阿里巴巴服务器上已经处理过的数据量已超过 100PB，相当于 580 亿本藏书。互联网搜索巨头百度所有服务器上存储的数据量在 2013 年已经接近 EB 级别。

大规模数据的爆发式产生对其存储有着新需求。各种数据具有不尽相同的价值量，甚至某数据在不同时期价值量也有着巨大差异；然而，任何数据的使用频率都会随着产生时间的增长而日渐下降。根据研究表明，企业存储的数据量只有 20%会产生较高频率的访问，而其它 80%的数据量并不会被频繁访问使用，这就为数据的

差异化存储提供了可能管理方案。利用不同存储介质的读写性能、价格、存储生命周期等不同特性为海量数据提供不同的存储空间，即分级存储管理技术可有效平衡企业存储海量数据的需求与成本控制的需求。分级存储管理的核心思想主要是根据数据访问频率等特性的不同，将其存储到不同特性的介质上，并通过软件自动转存各存储介质上的数据。

对具有较低访问频率的数据以归档、备份的形式进行存储对企业具有重要的数据安全意义以及经济价值。自然因素、人为因素都可以引起存储设备发生故障而损坏，从而给企业带来无法挽回的损失；数据归档、备份是抵抗数据损坏丢失风险最为行之有效的方法，所以数据归档、备份对一个企业来说显得极为重要。在海量数据的归档存储方面，光盘库有其显著优势所在。

光盘库是一种使用几百张甚至几千张光盘作为存储介质，并附带有抓取光盘所用机械手以及读写光盘所用光盘驱动器的光盘存储设备^[1]。相比传统的海量数据归档备份设备磁带库而言，光盘库在提供同样的数据可靠性、安全性的情况，对物理环境的要求要低很多；与此同时，光盘库的保存年限更长（约 50 年），读取使用次数更多，读取随机进行，日常运行能源消耗更低，更重要的是其维护非常简单。对于一次写的光盘而言，所存储的数据不会遭受网络病毒、黑客攻击等的破坏而出现丢失的情况，安全性非常高；而当前单张商用蓝光光盘的存储空间已达 100GB，使得数据存储密度得到大幅度提高。光盘库具有使用操作简单、海量存储空间、高可靠性的特点，同时广泛地支持各种网络通信协议，非常符合海量数据归档存储的特有需求。因此，本课题组确立了光盘库项目，旨在探索光盘库存储系统在海量数据归档存储场景的应用。

目前，存储系统在容量急剧增长及应用场景多元化的同时，其可靠性也面临着严峻挑战，纠删码作为存储系统容错的重要方法其重要性日益凸显^[2-4]。本课题研究在光盘库存储系统中采用纠删码容错技术，以期显著提高系统整体的可靠性。但是相比磁盘阵列冗余编码，光盘冗余编码优先需要更高的纠错能力，需要从磁盘缓冲区读取数据进行冗余编码计算，以及磁盘缓冲区空间容量相比光盘冗余编码盘组容量要小得多等限制。本文主要基于纠删码高容错能力、高空间利用效率等优势，针

对光盘纠错的特殊需要，设计符合光盘纠错需求的纠删码纠错方案。

1.2 纠错技术研究现状

移动互联网技术、大数据技术等取得显著进步的同时，也正以指数式速度产生海量数据，这促使存储系统中相关设备的数量快速增加，种类也急剧复杂化，直接导致存储系统更容易出现存储设备损坏等故障。为了提高存储系统的可扩展性，数据的高可靠性、高可用性，迫切需要对数据进行冗余存储以提高存储系统的纠错能力。目前存储系统增加冗余内容的方法主要有完全副本方法和纠删码方法两种。

完全副本方法即为数据块添加多份拷贝来提升系统可靠性的方式。完全副本机制的优势在于易于实现，可靠性高，可在复杂的存储系统中稳定地、持续地运行；此外，全副本方法还可以提升云存储系统用户读写的性能，并具备减轻单节点集中访问压力的效果。完全副本机制的明显劣势在于存储空间利用率非常低，例如对于存储 N 个数据副本的系统，其空间利用率只有 $1/N$ ，这对于规模较大的存储系统成本太过高昂；此外，多份拷贝的同步会带来巨大的带宽负载等。

而对于使用纠删码机制的存储系统，对原始数据块进行编码得到冗余校验块，当存储的数据块或校验块出现失效时，可以利用未失效块内容进行失效块重构，从而提供数据的纠错能力。纠删码起始于通信领域数据传送过程中检错、纠错需要，随着存储系统规模日渐扩大，数据内容对可靠要求日渐提高，纠删码冗余纠错机制研究更加深入、应用更加广泛^[5]。纠删码在数据传输系统中主要运行编码算法以及解码算法，而在数据存储系统中则会更高频率地运行更新算法，因而其更新性能成为存储系统评估纠删码方法的核心指标。

纠删码根据冗余是否独立于数据条块存储而分为横式编码以及纵式编码。其中横式编码是指校验内容独立于数据内容存储的机制，即两者分别存储在单独条块内，同一条块中并不会混合存储冗余和数据，如容双错 EVENODD 编码算法以及容三错 STAR 编码算法等^[6-9]。而纵式编码是指校验内容与数据内容混合存储的机制，如 X-code 编码算法等^[10]。纵式编码的天然优势在于可以将编码计算开销相对均衡地分布到各个磁盘块上，从而从根本上解决横式编码高频率读写冗余盘造成的瓶颈问题；

然而，纵式编码的这种均衡特点使得数据块间更加直接地相互依赖，这直接导致该编码模式不易于扩展。

目前已经有多种比较成熟的纠删码算法，包括阵列码算法、RS（Reed-Solomon）类编码算法以及一些近年出现的新型编码算法等。其中，RS 编码于 1960 年由 Vandermonde 设计实现，并经过数十年的深入研究已经得到充分的发展，并具备了非常完善的理论基础以及具有各种特性的开源实现^[11-13]。RS 类编码算法可以分为 VRS（Vandermonde-based RS）编码算法和 CRS（Cauchy-based RS）编码算法两种；目前可以实现容纳任意错误数的编码方案只有 RS 类编码，即 MDS 编码（Maximum Distance Separable Codes）^[14-17]。而上面提到的传统纵式编码和横式编码大多各有其优缺点，目前针对结合纵式和横式编码两者的优点，以更好地发挥阵列编码的优势，正涌现出越来越多的研究，如 HoVer 编码、GRID 编码等^[18]。但是 MDS 编码的编码算法、解码算法的性能相对较低，并不能很好地满足对于实时编码有着较高需求的存储系统；相关研究已经通过牺牲少量的存储空间利用率来优化编解码性能，如 LDPC（Low-Density Parity-Check）编码算法，LDPC 编码是一种近似 MDS 编码，相比 MDS 编码其编码、解码需要较大的数据传输量，所以当网络带宽相比 CPU 带宽较高时，LDPC 编码会比 MDS 编码有着更好的性能，但是当网络带宽较低时则相反^[19,20]。纠删码优劣的评估指标包括容错能力、计算效率、存储空间利用率三个方面，这三个方面直接影响存储系统的可靠性、访问性能、存储成本三个指标，传统编码通常很难同时在三个方面都达到最优状态。

相比完全副本机制，纠删码机制具备更高的空间利用率、更强的纠错能力等优点，逐渐被规模不断扩大的存储系统所广泛采用，例如 RAID、Windows Azure Storage 以及 OceanStore 等^[21-23]。但是各种数据存储系统对冗余纠错方法有着各不相同的需要。例如磁盘阵列数据存储系统是由众多的原始数据磁盘以及少量的编码磁盘组成，运行频率较高的是编码算法，尤其是更新算法，所以阵列码更能满足其需求特点，阵列码中最典型的纠错机制是 RAID^[24,25]。P2P（Point to Point）系统具有数量庞大的磁盘存储设备，该类系统具备廉价性、高流动性等特点，且存储设备损坏或者脱网的几率很高，故需要具备较高纠错能力的冗余机制，所以完全副本机制更能满足其

需求特点。分布式文件系统对数据存储具有高可用性要求，所以解码算法性能的高低是冗余机制选取关键所在。而归档备份类型的数据存储系统主要对数据可靠性有着非常高的要求，所以该类系统需要具备较高容错码率、容错能力的纠删码^[26-28]。

1.3 本文内容与安排

本文的具体内容和结构安排如下：

第一章绪论对本课题的来源、目的、意义和冗余纠错技术研究现状进行简介。

第二章是对光盘库多盘片冗余纠错系统实现所要用到的相关理论技术进行介绍，具体包括存储系统中纠删码的相关概念，典型冗余纠错算法以及若干近年新型纠删码算法的研究情况。

第三章是光盘库多盘片冗余纠错系统的总体设计，对系统的整体架构进行介绍，并进行需求分析，针对这些进而对系统软件进行总体设计和功能模块的划分，对系统各子模块进行概要设计。

第四章是系统关键模块和算法的研究实现，并详细介绍系统若干主要模块的详细工作流程及其实现方法。

第五章是光盘库多盘片冗余纠错系统的测试，包括系统的功能完整性测试，以及关键算法性能优化的效果测试。

第六章总结全文内容，指出几个尚待解决的问题，并展望冗余纠错系统未来探索的方向。

2 冗余纠错基本理论与技术

本章将主要阐述冗余纠错基本理论以及相关技术，具体地包括纠删码涉及的相关概念，典型常用纠删码的概要分析和研究，以及各种纠删码优缺点的分析和总结。

2.1 纠删码的相关概念

纠删码已较为广泛的用于规模较大的存储系统来提升系统容错能力以及系统性能，例如广域存储平台、P2P 存储平台以及归档存储系统等^[29,30]。

假设目前有 k 个数据存储设备，每个设备容量大小为 S ，然后为这 k 个设备添加 m 个校验存储设备，每个设备容量大小同为 S ，任一存储设备的失效都可能导致数据内容不可读取。纠删码的核心思想在于如何将这 k 个数据存储设备通过一定的编码运算，生成 m 个校验存储设备；而当任意不超过 m 个存储设备发生故障时，都可以利用仍可读取的存储设备，通过一定的解码运算重构故障设备。当前纠删码基本概念并未实现一致定义，故下面将说明本文涉及的相关术语^[31-33]。

数据是指用户产生并由系统存储的真实内容。冗余是指通过某纠删码算法，计算数据产生的校验信息串，其目的在于提供容错能力。

元素是纠删码计算的基本单元，且不可再分。条带是指独立组成一次编码计算的集合，即类似该纠删码的一个具体实现，不同条带间具有完全独立性；此外，由于条带在逻辑上并不与物理地址绑定，所以可以用轮换法应对存储设备内容更新引起的负载不均衡问题，但由于光盘盘片并没有反复更新的需要，光盘上应用纠删码时不需要轮转存储。条块是指某一条带内某个存储设备中所有元素的集合，其容量等于元素个数乘以元素容量。图 2.1 是条带组成示意图，其展示了一个条带是如何由元素和条块组合而成的。

MDS 编码是指符合 Singleton 边界条件的校验计算方式，该方式在空间利用率方面能够达到理论最优值；简单来说，MDS 编码方案的容错能力等于校验存储设备的个数。

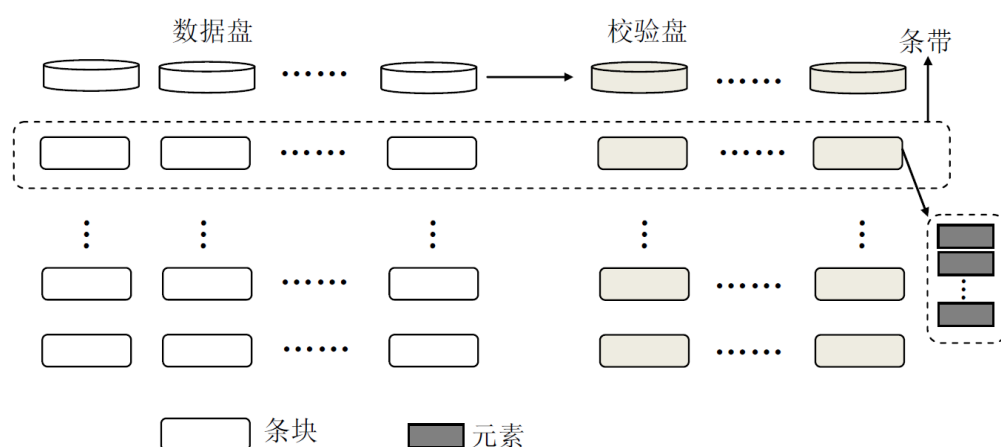


图 2.1 条带组成示意图

编码是指在纠删码算法下由实际内容计算出校验内容的过程。更新是指在数据内容发生更改后，依据编码算法再次计算与其相对应校验内容的过程；普通存储系统更改数据内容的频率比较高，所以更新性能是评估纠删码算法的核心指标；但是在光盘存储系统中，用于归档存储的光盘盘片只能一次写，所以该场景下并不需要用更新性能来评价纠删码算法。解码是指在某存储设备发生故障失效后，使用数据重构算法通过未失效存储设备恢复出失效设备存储内容的过程。

2.2 经典冗余纠错理论

本节将对各种经典的纠删码理论及其优缺点展开分析和总结。

2.2.1 RS 类编码

目前能够实现任意数据存储设备数目 k 和校验存储设备数目 m 的 MDS 算法只有 RS 类编码。RS 类编码经过几十年的研究，已具备比较完善的理论基础^[34-36]。RS 类编码计算通过 $GF(2^w)$ 域中元素多项式运算实现，具体包括加、减、乘、除等算术运算^[37]。RS 类算法包括 VRS 编码算法和 CRS 编码算法；VRS 算法所用编码系数为范德蒙矩阵（Vandermonde Maxtrix，即 VM），通过 $GF(2^w)$ 域中多项式运算实现；而 CRS 算法所用编码系数是柯西矩阵（Cauchy Matrix，即 CM），利用 $GF(2)$ 执行异或运算。

1. VRS 编码

VRS 编码的核心是通过 Vandermonde 生成矩阵与数据块向量相乘得到冗余校验向量的，假设定义数据字的线性组合函数 F_i 如公式 2.1 所示。

$$c_i = F_i(d_1, d_2, \dots, d_n) = \sum_{j=1}^n d_j f_{i,j} \quad (2.1)$$

VRS 编码的重构算法核心是通过未失效数据块及校验块对应的子生成矩阵的逆矩阵与有效的块内容向量相乘得到故障块内容。VRS 编码重构算法实现的关键条件是有效数据块及校验块所对应的子生成矩阵在 $GF(2^w)$ 域上必须要满足可逆的条件。为保证 VRS 编码可恢复任意不超过 m 个数据块或者校验块,生成矩阵的任意 k 行子矩阵需要在域 $GF(2^w)$ 上是可逆的。线性代数上广泛使用的 VM 完全可以满足 VRS 编码对失效信息块重构的矩阵属性要求。具体通过对 VM 使用基本初等变换使前 k 行方阵转化为单位阵,从而构建 VRS 算法需要的编码系数。具体 VRS 算法原理如图 2.2 所示。

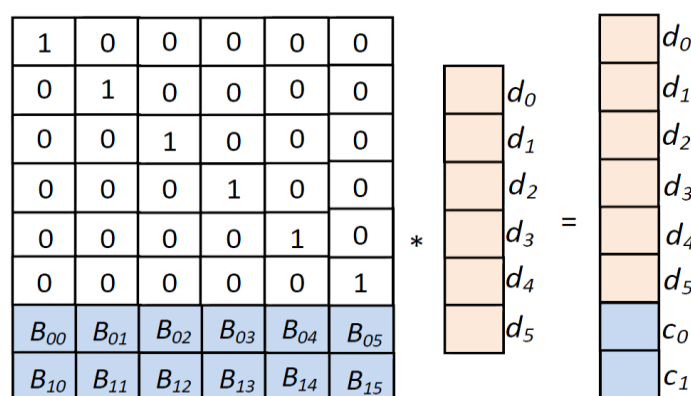


图 2.2 VRS 编码原理示意图

VRS 编码计算在 $GF(2^w)$ 中进行,加法为异或计算,执行速度比较快;但是,乘法运算相比异或运算复杂度要高很多,一般通过离散对数计算及逆对数表来实现。因此,在目前 CPU 结构下,VRS 编码即标准 RS 编码实现开销过大,难以适应具有高性能要求存储系统。

2. CRS 编码

CRS 编码是基于标准 RS 编码发展而来,是 RS 类编码中具有最高性能的一种实现方式。VRS 算法所有计算必须在 $GF(2^w)$ 中执行,开销过大的乘法以及子矩阵求逆计算是 VRS 算法性能提升的巨大阻碍;此外,VRS 编码的元素位数需为 w 位,即元素最大值不超过 2^w , w 典型值如 4、8、16 等,其运算并不能充分发挥 32 位、64 位等 CPU 的计算性能。

相比 VRS 编码, CRS 编码在编码模式上有两方面改进。首个改进是采用 m 行 k 列的 CM, 其元素同样在 $GF(2^w)$ 中且满足条件 $k+m \leq 2^w$; 但是 w 值可以任意选择, 因为 w 对矩阵元素位数并无限制, 这是柯西矩阵的特性。柯西矩阵定义如下: 设 F 是一个域, 且 $\{x_1, x_2, \dots, x_m\}$ 、 $\{y_1, y_2, \dots, y_k\}$ 是 F 中元素的两个集合, 分别记为 X 和 Y , 并满足如下两个条件: 第一, 对于 $\forall i \in \{1, 2, \dots, m\}$, $\forall j \in \{1, 2, \dots, k\}$ 满足不等式 $x_i + y_j \neq 0$, 即 $x_i \neq y_j$; 第二, 对于 $\forall i, j \in \{1, 2, \dots, m\}$, $i \neq j$ 满足不等式 $x_i \neq x_j$, 且对于 $\forall i, j \in \{1, 2, \dots, k\}$, $i \neq j$ 满足不等式 $y_i \neq y_j$; 那么, 如图 2.3 所示的矩阵是域 F 上的柯西矩阵。

$$\begin{bmatrix} \frac{1}{x_1 + y_1} & \frac{1}{x_1 + y_2} & \dots & \frac{1}{x_1 + y_n} \\ \frac{1}{x_2 + y_1} & \frac{1}{x_2 + y_2} & \dots & \frac{1}{x_2 + y_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{1}{x_m + y_1} & \frac{1}{x_m + y_2} & \dots & \frac{1}{x_m + y_n} \end{bmatrix}$$

图 2.3 柯西矩阵示意图

柯西矩阵的任何子方阵都是一个非奇异矩阵, 因为柯西矩阵是一个满秩矩阵, 而满秩矩阵是一个矩阵可逆的充分必要条件。所以, 如果矩阵 C 是域 $GF(2^w)$ 中 m 行 k 列的 CM, 那么 $(I_m|C)$ 是 CRS 算法的编码系数矩阵。

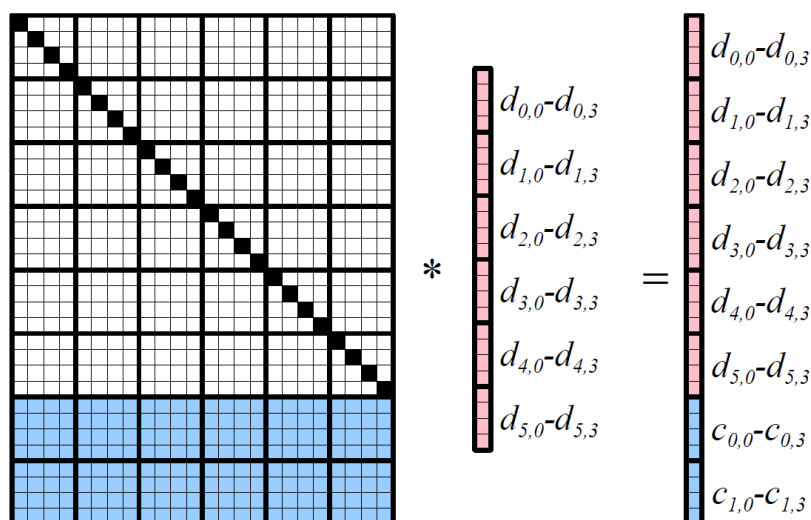


图 2.4 CRS 编码原理示意图

CRS 算法第二个改进是将 $GF(2^w)$ 中乘法计算转化成二进制位乘法计算即异或计算；其原理为 $GF(2^w)$ 域上任意数值都能够利用 $1 \times w$ 位列向量表达，也能够利用 $w \times w$ 位矩阵表达，从而将域上乘法直接转换成简单得多的异或计算。图 2.4 是 CRS 编码原理示意图。分析 CRS 编码的运算过程可发现，其执行效率首先取决于柯西矩阵位表示方式中矩阵元素 1 的数目，这将是 CRS 编码性能提升的关键所在^[38]。

2.2.2 横式阵列码

阵列码 (Array Code) 是完全基于异或计算的编码模式，其基本思想在于将数据块和校验块一起存储在二维或更高维数的阵列中，从而能够基于异或运算实现更简易的编码过程、更新过程以及解码过程，故其应用最为广泛。

阵列码按数据块与校验块间存储关系可分为横式阵列码与纵式阵列码两种。横式阵列码的特点在于将校验块与数据块分开存储于各自存储设备中，其优势在于存储系统易于扩展编码规模，其劣势在于容错能力一般较低。例如 RDP 算法、Liberation 算法等横式阵列码的纠错数量不超过两个^[39]。近些年也出现了一些容错能力大于 2 的横式阵列码，如 Star 编码等。

这类横式阵列码更大的局限性在于对存储设备的数目有着苛刻的要求，如存储设备的数量需要是素数或者素数减去 1，甚至需要任一条块内元素数量必须与存储设备数量相对应。但这类横式阵列码在存储空间利用率上可以达到理论最优，故属于 MDS 编码。

从数学的角度观察所有的编码算法，其异或系数能够用生成矩阵的方式进行表达；所以，某种具体的纠删码算法的编码性能和更新性能可以直接通过其对应的生成矩阵中元素 1 的个数体现出来，相关研究就可以考虑减少矩阵中 1 个数的具体方法和策略来提升纠删码的性能。

2.2.3 纵式阵列码

纵式阵列码的特点在于将校验块与数据块进行混合存储，故某一条块内可能在存有数据内容的同时还存有校验内容。纵式阵列码的优势在于矩阵几何构造简单，可以使异或编码运算和更新运算开销可以相对均衡地分布在各个存储设备上，从而

避免横式阵列码存储设备读写热点过度集中带来的严重瓶颈问题。

纵式阵列码的读写负载均衡的优势，也给其带来了一定局限性；这主要在于数据块和校验块的混合分布，使得存储设备间的依赖度非常高，即不易于编码组规模的扩充。

近些年来在横式阵列码领域出现了多种优秀的编码方案，例如 X-code 编码 P-code 编码、WEAVER 编码等算法^[40]。X-code 算法按存储空间利用率来讲属于 MDS 编码，不同于大多数横式阵列码，它在编码性能、更新性能、解码性能等多个方面都可达到理论最优值，其缺点也是在于作为纵式阵列码数据块和校验块混合存储带来的强依赖性，不易于存储系统规模的扩充。

X-code 算法另一局限性在于对存储设备数量 k 的限制十分苛刻，需要 k 必为素数，才可实现任意两个存储设备失效的场景下都可以重构所有失效设备。同时，由于 X-code 编码模式中校验元素与数据元素的高度依赖性，使其难以基于该编码设计出容错能力更强的编码方案。最后，X-code 编码的一个严重局限性是其重构算法必须严格按照特定的顺序执行，所以，该算法难以并发执行解码过程，重构性能提升困难。但全面来评价，X-code 算法依然不失为一种非常出色的纠删码模式。

2.2.4 LDPC 编码

LDPC 编码是一种所有运算步骤都是基于异或运算的编码模式。LDPC 编码并不属于 MDS 编码的范畴，而是一种近似空间利用率最优的设计方案；LDPC 编码在存储空间利用率上进行少量的降低，换取编码效率的大幅提升，其编码速度远快于大多数 MDS 类编码。

LDPC 编码的基本运算原理是基于一个二部图实现的，例如典型的基于 Tanner 图的编码模式；该模式利用 Tanner 图左端包含全部的数据块和校验块，右端节点用以表示两种块内容组成的异或关系式，其含义为与该节点相连的左端各信息块内容的异或结果等于 0。LDPC 算法在 Tanner 图模式下编码过程如图 2.5 所示，据此可以看出该模式下的编码过程非常灵活，尤其是在重构过程中，算法可统一对待数据块与校验块，作为同一种元素的方式直接采用二部图原理为基础执行恢复过程。

LDPC 算法的突出优势在于重构过程简单、性能较高。

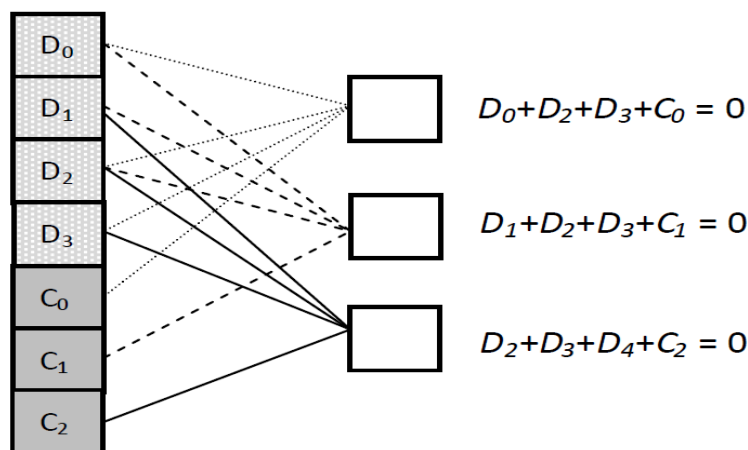


图 2.5 LDPC 编码示意图

但全面评价来看，LDPC 算法当前仍处于理论探索过程中，实际应用非常少，其关键原因在于实现 LDPC 编码非常困难，且 LDPC 算法利用 Tanner 图实现的方式多为不规则编码。所以，LDPC 编码仍需要进一步的研究才能满足实际应用需求。

2.3 近年新型纠删码理论

上一节对经典的几类编码方案进行了分析研究，本节将对近年出现的新型纠删码理论展开探讨。例如，上一节涉及的横纵两种阵列码，各自皆有优点，也皆有缺点，探索将两种编码模式的优点充分结合，设计出具有新特性的编码方案成为非常有意义的研究方向；且该领域内已经出现少量新型编码方案，如 HoVer 编码等。

此外，前述经典纠删码理论全部使用元素作为运算单元，其局限性在于难以在纠错能力、编解码性能、存储空间利用效率方面同时达到理论最优值。GRID 编码就是以此为出发点，采用条块作为异或运算的基本单元，有效地解决了上述问题，其存储空间利用率甚至可以超过 80%；此外，其实现方法相对也不复杂。

1. Pyramid Codes

RS 类编码是一种 MDS 编码，可以达到存储空间利用率理论最优值，在云存储系统中有效降低单位空间的成本；但其解码算法的执行速度相对较低，因为任意存储设备的失效重构，都需要所有 k 个存储设备参与失效设备的重构。Windows Azure

存储系统基于 RS 类编码，设计了 Pyramid Codes 即金字塔编码，这是因为其校验块间逻辑关系类似金字塔递进式结构，其冗余率范围大多在 1.3~1.5 之间。

图 2.6 是金字塔编码示意图，它首先选取某个 RS 算法，将其某个校验块分割为两个局部校验块，图中 $C_{1,1}$ 只对 $d_1 \sim d_6$ 编码， $C_{1,2}$ 则只对 $d_7 \sim d_{12}$ 编码，这样就基于 RS_{12+3} 编码构建出了 $Pyramid_{12+4}$ 编码。存储空间利用率比 RS_{12+3} 略高，同样可以容忍 3 个任意的错误。对于解码算法，如果同时发生三个错误，且都在同一组内，此时可以利用 $C_{1,1}$ 与 $C_{1,2}$ 的异或值恢复为 C_1 ，进行 RS_{12+3} 运算即可进行损坏盘重构。单盘损坏重构代价从 12 变为 6，这在磁盘等存储系统实际运行时更多的是单盘失效的情况下，具有重大意义。

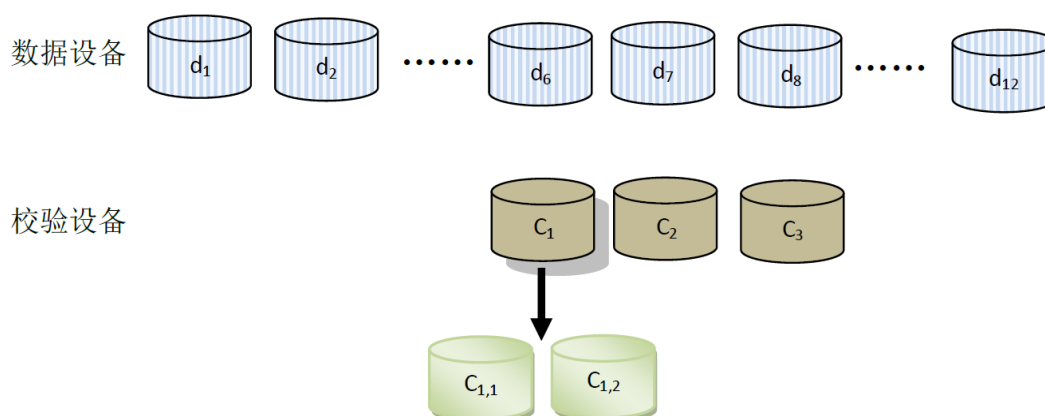


图 2.6 Pyramid Codes 编码示意图

2. 混合容错编码

设想这样一种存储系统解码场景，目前只有一个存储设备损坏，而当重构该损坏设备时，在其他设备发生某个扇区错误；对于这种情况，RAID5 编码将不能恢复损坏的数据，从而导致数据丢失；如果使用 RAID6 编码方案，则可以实现损坏设备重构，但所付出代价过于高昂^[41]。另一方面，随着 SSD 更多的使用且 SSD 相比 HDD 的单价更高，这种情况下 RAID6 所付出的成本过于高昂。所以研究人员考虑更具存储效率的编码方案，使得编码模式具备扇区错误和设备错误混合容错的能力。目前新提出的 SD (Sector-Disk) 纠删码及其改进 STAIR 编码都是这类编码^[42,43]。混合容错编码适用于扇区错误和设备错误都会发生的场景，光盘存储介质故障类型基本为整个盘片损坏；所以，光盘存储介质并不需要采用混合容错编码方案。

2.4 本章小结

本章首先简单介绍了数据纠错技术相关的基本概念，然后对 RS 类编码、纵横阵列码、LDPC 算法等经典冗余纠错理论和近年出现的新型纠错码进行了概要介绍，并对相关算法的优缺点进行了分析和总结。

3 冗余纠错系统总体设计

本章将主要针对光盘库存储系统对可靠性的需求展开分析，以得到光盘库冗余纠错系统的主要需求；根据系统需求展开对冗余纠错系统与光盘库存储系统其它模块接口的设计、冗余纠错系统的概要设计以及冗余纠错系统各子功能模块的设计。

3.1 典型应用的需求分析

充分挖掘系统的需求，既包括基本功能需求，又包括特有功能需求，利于更好把握系统的整体结构、关键特性，从而更好地制定策略、制定系统设计方案，以为系统功能的完备性、关键需求的满足提供充分的保障。

本文所要设计的光盘库冗余纠错系统，概要来说是为光盘库存储系统提供冗余纠错能力，提高光盘库存储系统的整体可靠性；具体来说，为相关数据存储光盘添加校验信息，从而使得整个光盘库存储系统在发生一定范围内、一定数量的光盘盘片失效时，系统仍然可以利用所添加的冗余校验光盘信息，重构失效光盘的数据内容，并将其存储到新的光盘盘片中，或者直接为数据请求提供服务，从而为光盘库存储系统提供必要的纠错能力，提高光盘库存储系统的整体可靠性。光盘库存储系统对纠错能力具有以下几个方面的功能需求以及性能需求：

1. 存储系统基本纠错功能需求。光盘库作为存储系统的一种，需要其数据内容具备可靠性、可用性。假设有 k 张数据光盘，依次记为 $D_1, D_2, \dots, D_k$ ，以及 m 张校验光盘，依次记为 $C_1, C_2, \dots, C_m$ ，每张校验光盘的内容通过数据光盘的内容计算得到；光盘库存储系统对于纠错能力的要求是定义每个 C_i 的计算方式，使得 $D_1, D_2, \dots, D_k, C_1, C_2, \dots, C_m$ 这 $k+m$ 张光盘盘片中，如果有不超过 m 张盘片失效，则其内容可以通过未失效光盘重构。

2. 归档系统独有纠错功能需要。上述关于存储系统普通纠错功能的描述中，不同的编码算法添加 m 张冗余光盘所能恢复的失效盘数量是不同，需要依据存储系统具体功能特点选择编码算法。光盘库作为归档存储系统类型的一种，光盘库冗余纠错子系统有其不同的特点与需求。归档类型存储系统首先需要实现高容错能力以及

高存储空间利用率；归档存储企业内容虽然大多不频繁访问，但随着存储周期延长，一旦需要这些数据时，其价值往往是不可替代的，所以归档类型存储系统首先要具有高容错能力，即需要较多校验光盘以容较多的光盘故障。

此外，归档系统所存储数据内容的生命周期往往比较长，所以对编码方案的运算性能并没有太苛刻的要求；而另一方面，在相同的存储可靠性条件下，采用低码率编码方案可以有效地减少冗余盘比率，从而有效降低整个存储系统的成本。所以，归档系统适合采用 MDS 编码作为其编码方案，因为 MDS 编码是相同码率下可靠性最高的编码方案，也即相同可靠性指标下码率最低的编码方案。而唯一对校验设备数量无限制的 MDS 编码是 RS 类编码，故本系统将基于 CRS 编码实现。

3. 光盘库特有纠错功能需求。光盘库存储系统采用光盘作为数据存储介质，单张光盘容量较大，目前商用蓝光光盘单盘容量已达 100GB；通用的光盘数据存储方式，通常需要先将数据准备好再进行单盘不间断刻录，如组织成 ISO 文件，从而避免单盘多次追加写入造成较大空间浪费。在归档系统高容错能力要求前提下，往往需要较大的编码组规模，较多的冗余盘数量，所以这种采用光盘连续刻录的方式，会使得单个编码分组需要较大磁盘暂存空间开销；另一方面，光盘数据内容是由客户端逐渐上传的，并非较短的时间内快速产生。所以，将部分已生成的光盘文件提前进行编码，不仅可以提高数据内容在刻录光盘前的可靠性，又可减轻盘组整体编码对存储系统服务器端产生长周期的高负载压力。在数据重构时，受限于光驱数量及系统负载等因素，也有逐部分解码的功能需要。

4. MDS 编码性能优化需求。具有高容错能力的 MDS 编码中，对盘组规模、盘组码率没有限制的编码只有 RS 类编码，RS 类编码基于生成矩阵进行编码，在编码组内以条带为编码单元。首先，这类编码在条带内部通常有许多数据内容进行同样的异或运算，所以可充分利用已计算的校验信息进行编码，从而有效减少重复异或运算，以降低光盘库存储系统的整体负载。第二，这类编码方案在条带之间具有无关性，可以充分利用多线程技术并发执行编解码运算，以提高编解码速度^[44]。最后，由于光盘库存储介质的一次写特性，并不会对已写入的数据内容进行物理上的更改，所以并不需要考虑冗余纠错算法的更新开销。

5. 文件编码读写性能需求。作为光盘库存储系统，冗余纠错系统是通过为蓝光光盘 ISO 文件添加冗余校验信息提供纠错能力的，这种方式不同于磁盘阵列、SSD 等存储方案对内存中数据直接编码，然后将其写入相应存储设备；光盘冗余计算数据来源于磁盘中的 ISO 文件，所以有更多磁盘或 SSD 缓存空间读写；故需要充分评估并优化编解码过程磁盘或 SSD 缓存空间 IO 的性能。

6. 编码方案的可配置性。光盘库存储系统所存储的不同数据具有不同的重要程度，可采用不同的码率方案，设定不同的编码组规模，从而提供不同的纠错能力，实现差异化的存储容错成本；这就需要冗余纠错系统提供可供配置的数据盘数目和编码盘数目选项，从而提高存储系统的灵活性，以最大化降低存储总成本。

3.2 冗余纠错系统接口设计

冗余纠错系统与光盘库存储系统多个模块之间有着直接的交互，需要设计良好的接口，以应对不同的功能需求，并为整个存储系统提供较高的可扩展性。冗余纠错子系统主要与光盘库系统管理模块、刻盘子系统以及读盘子系统有系列交互，需要进行相关接口的设计；具体地，可以分为数据编码功能接口设计以及数据解码功能接口设计。

冗余纠错子系统编码功能主要接收刻盘子系统发送的待刻盘数据文件列表，生成冗余校验文件，并将结果返回给刻盘子系统；冗余纠错系统解码功能主要接收读盘子系统发送的待重构数据文件列表，通过解码算法重构数据文件和校验文件，将结果返回给读盘子系统，并将重构的光盘文件作为刻盘任务发送给刻盘子系统。图 3.1 是冗余纠错子系统与光盘库存储系统其它模块间接口设计。

1. 编码功能接口设计

客户端将数据上传至光盘库服务器端，在服务器端由系统管理模块组织生成文件；刻盘子系统将生成的待刻录文件列表及编码方案参数发送给本冗余纠错子系统，编码方案参数包括一个编码组内数据盘数量 k 和校验盘数量 m 等。如果刻盘子系统所发送的文件数量等于 k ，则冗余纠错子系统将收到的所有数据文件执行一次完整编码过程；如果所发送的 ISO 文件数量大于 k ，则冗余纠错子系统将先选取 k 个数据文

3.3 冗余纠错系统概要设计

前面两节已经对光盘库冗余纠错系统的功能和性能需求进行了分析，并对冗余纠错系统与光盘库存储系统其它功能模块的接口进行了设计，本节将针对本纠错系统需求以及对外接口情况，展开系统概要设计。

光盘库纠错系统的核心功能在于给定 k 个数据光盘文件的情况下，对其数据进行异或编码，生成 m 个校验光盘文件；在该组 $k+m$ 个光盘盘中发生任意不超过 m 张光盘失效的情况下，所有失效光盘文件内容都可以通过解码算法实现数据重构。依据该核心功能需求，现将冗余纠错系统总体划分为五个功能子模块，依次包括基本编码模块、基本解码模块、编解码分段模块、编解码优化模块以及编解码并发模块，其中编解码分段模块、编解码优化模块以及编解码并发模块为冗余纠错系统的公共任务处理模块，同时为基本编码模块、基本解码模块提供计算服务。图 3.2 为冗余纠错系统总体设计图。

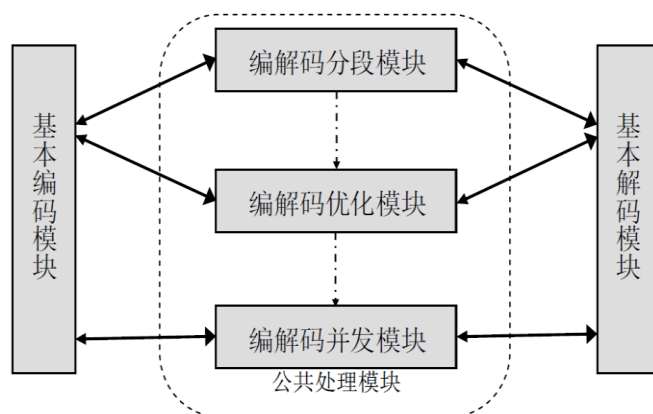


图 3.2 冗余纠错系统总体设计图

各个子模块的主要功能以及调用关系如下所述。

1. 基本编码模块主要负责与刻录子系统通信，获得待刻录数据文件列表及编码方案参数等信息；然后通过调用编解码分段模块、编解码优化模块以及编解码并发模块等执行一次编码过程；完成编码运算后，将数据文件列表及校验文件列表返回刻录子系统。

2. 基本解码模块主要负责与读盘子系统通信，获得失效数据所在光盘编码组

的元数据文件等信息；然后检测、分析该编码组内所有失效光盘盘片，如果故障数量不超过容错数量，则通过调用编解码分段模块、编解码优化模块以及编解码并发模块等执行一次解码过程；最后将数据重构的结果或者不可重构的信息返回给读盘子系统，并将重构成功的光盘文件发送至刻录子系统，以将其写入新的蓝光光盘。

3. 编解码分段模块主要负责为基本编码模块以及基本解码模块提供编解码矩阵分段计算的功能，从而使得一个编码组可以分为多个编解码阶段完成；该模块执行完成后，会将分段的生成矩阵返回给编解码模块。

4. 编解码优化模块主要负责为基本编码模块以及基本解码模块提供矩阵异或运算优化调度的功能，以有效减少编码算法以及解码算法的异或运算总量，从而加快编解码算法执行；该模块执行完成后，会将数据块异或运算的调度命令返回给编模块或者解码模块。

5. 编解码并发模块主要负责为基本编码模块以及基本解码模块提供 Linux 线程池的并发计算功能。该并发执行功能主要根据 CPU 核心数以及光盘库存储系统整体负载情况，通过将编解码运算相对独立的多个条带同时执行实现。

3.4 冗余纠错功能模块设计

上一节对冗余纠错系统进行了总体设计，本节将对系统两个基本功能模块以及三个公共服务功能模块进行设计和阐述。

3.4.1 基本编码模块设计

基本编码模块是编码生成校验光盘文件过程的核心调度模块，通过与编解码分段模块、编解码优化模块以及编解码并发模块的交互完成数据编码功能。基本编码模块编码过程包括编码方案参数检查配置及文件列表检查操作、编码二进制分配矩阵创建和优化操作、调用编解码分段模块分段处理编码矩阵操作、调用编解码优化模块生成数据块异或运算调度优化命令操作、读取数据文件至内存缓冲区操作、调用编解码并发模块并发执行编码操作、写入校验文件至磁盘操作等。图 3.3 是基本编码模块功能设计图。

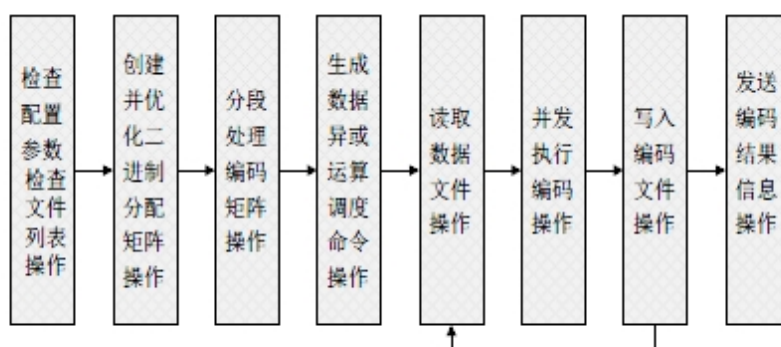


图 3.3 基本编码模块功能设计图

编码方案参数检查及配置主要是依据输入的数据文件个数以及预期的校验文件个数选择最合适的编码域 $GF(2^w)$ 即 w 值，将异或运算单元大小进行机器字对齐，将缓冲区大小进行单文件编码条带大小对齐，根据编码条带大小计算冗余校验文件大小，根据内存缓冲区空间大小计算文件读取次数等操作。文件列表检查操作主要是确认数据文件的有效性，并根据数据文件列表以及校验文件列表生成文件操作列表，从而统一地进行文件读取和写入操作。

编码矩阵创建操作简单，利用域 $GF(2^w)$ 的前 m 个值为数组 X 的元素，后续 k 个值为数组 Y 的元素，直接生成柯西矩阵。矩阵优化操作是通过一系列启发式矩阵基本初等运算，减少该柯西矩阵对应的二进制分配矩阵中 1 的个数，然后依据该柯西矩阵得到二进制分配矩阵 F 。

分段处理编码矩阵操作主要是根据输入的段数，设定一个编码组属于各段的数据文件数目，然后据此将之前生成的矩阵 F 分割成若干段，并将不属于某段的矩阵元素全部赋值为 0，以简化后续异或运算调度命令生成操作。

生成数据块异或运算调度命令操作主要是根据上述各段矩阵，生成数据块异或运算的命令，该生成过程会充分利用已生成的相同异或运算结果减少异或运算次数。

读取文件过程在首段编码时，只需读取数据文件至内存缓冲区。后续各段编码时，一方面，需要读入该段范围内数据文件至内存缓冲区；另一方面由于冗余校验文件中已存有部分校验内容，所以也需要读至内存缓冲区中。

并发执行编码操作主要是利用异或运算调度命令，对读入内存缓冲区的数据内容以及校验内容并发执行异或运算，该并发运算通过 Linux 线程池实现。写入编码

文件操作将生成的冗余校验内容写入校验文件。

最后，待所有数据文件或某段内数据文件完成编码任务时，向刻盘子系统发送编码结果信息；同时，将编码组编码方案的相关参数写入编码组元数据文件。

3.4.2 基本解码模块设计

基本解码模块是解码过程中的核心调度模块，通过与编解码分段模块、编解码优化模块以及编解码并发模块的交互完成数据重构功能。基本解码模块解码过程包括读取并分析编码方案元数据文件操作、检测并创建有效及失效光盘文件列表操作、创建编码二进制分配矩阵操作、创建解码二进制分配矩阵操作、调用编解码分段模块分段处理解码矩阵操作、调用编解码优化模块生成数据块异或运算调度命令操作、读取参与解码有效文件至内存缓冲区操作、调用编解码并发模块并发执行解码操作、写入重构文件至磁盘操作以及发送信息至读盘子系统和刻盘子系统等。图 3.4 是基本解码模块功能设计图。

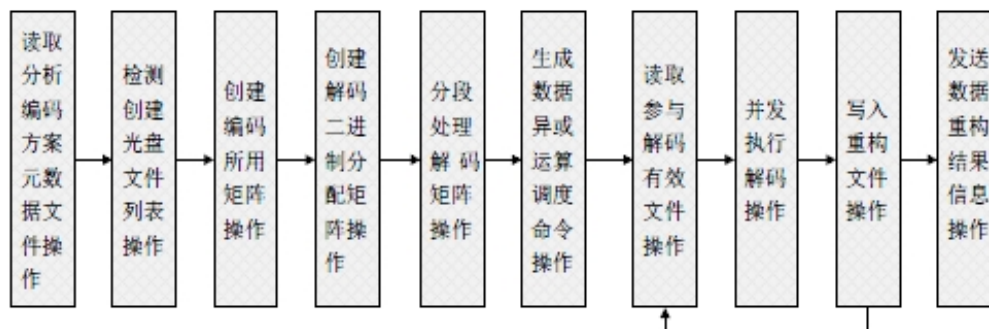


图 3.4 基本解码模块功能设计图

读取、分析编码方案元数据文件操作主要负责检测该编码组内所有光盘盘片，如果故障数量不超过容错数量，则执行后续数据重构操作；否则，将不可重构的信息返回给读盘子系统。如果可以重构失效光盘，则创建参与失效光盘重建文件列表以及失效光盘文件列表；然后，依据编码方案参数创建编码矩阵，并据此生成解码二进制分配矩阵 F' 。

分段处理解码矩阵操作主要是根据输入的段数，设定一个解码组属于各段的数据文件数目，然后根据各段文件个数将 F' 分为多段，并把不在该段范围内的元素全部初始化为零，以简化后续异或运算调度命令生成操作。

生成数据块异或运算调度命令操作主要是根据上述各段矩阵生成数据块异或运算的命令，该算法同编码过程异或运算优化调度算法相同。

读取参与解码有效文件操作在首段解码时，只需读取有效文件至内存缓冲区。后续各段解码时，一方面，同样需要读入该段范围内有效文件至内存缓冲区；另一方面由于失效文件中已存有部分解码内容，所以也需将其读至内存缓冲区中。

并发执行解码操作主要是利用生成的异或运算调度命令对读入内存缓冲区的文件内容并发执行异或运算，该并发运算方法同编码过程 Linux 线程池方法。写入重构文件操作将重构内容写入新文件。

最后，待所有失效文件或某段内失效文件都完成重构任务时，向读盘子系统发送解码结果信息，并向刻盘子系统发送新的刻录任务，由刻盘子系统将重构的文件写入新光盘。

3.4.3 编解码分段模块设计

编解码分段模块主要负责为基本编码模块以及基本解码模块提供编解码矩阵分段计算的功能，从而使得一个编码组可以通过多阶段完成；该模块执行完成后，会将分段的二进制分配矩阵返回给基本编码模块或者基本解码模块。

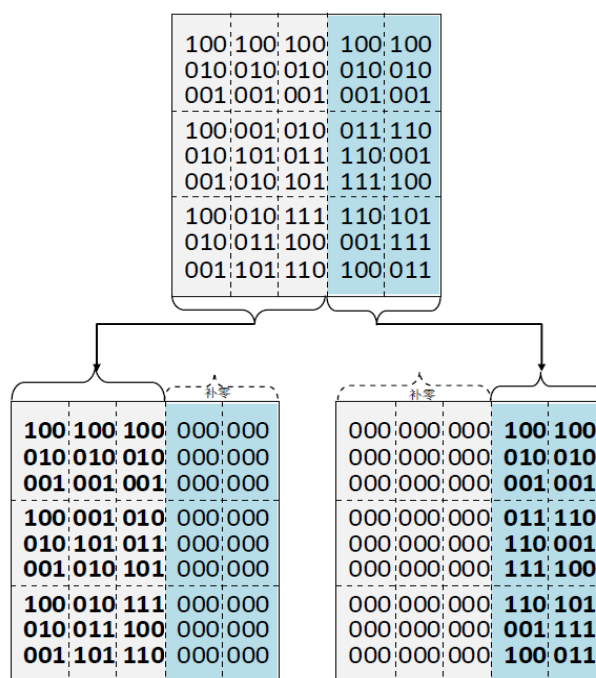


图 3.5 二进制位矩阵分段示意图

该模块功能通过提取矩阵各超级列数据元素，然后将不属于段范围内的超级列元素全部赋值为 0，这样便于根据元素 1 所在矩阵位置，计算出其对应的数据块所在光盘文件及文件内部字节偏移量。

图 3.5 是二进制位矩阵分段示意图。该编码矩阵以 $k=3$ ， $m=3$ ， $w=3$ 为例进行说明；第一段矩阵包含二进制分配矩阵前三超级列的元素，所以将处理前三个数据文件的编码运算；第二段矩阵包含二进制分配矩阵后续两超级列的元素，将处理后续两个文件的编码运算。

3.4.4 编解码优化模块设计

编解码优化模块主要负责为基本编码模块以及基本解码模块提供矩阵异或运算优化调度的功能；该优化算法的基础在于，CRS 编码单个编码字所需的异或运算次数往往大于理论最优次数，亦即存在一定量的数据块异或运算会被重复地执行；所以，设计相应算法以充分利用数据块公共异或运算结果，可以有效地减少异或运算的总次数；这不仅可以加快编解码速度，又可以降低光盘库存储系统的整体运算负载；该模块执行完成后，会将数据块异或运算的优化调度命令返回给基本编码模块或者基本解码模块。

以图 3.5 中二进制分配矩阵为例，记行号范围为 $[0, 8]$ ，行 5 和行 6 的公共 1 最长子序列为 $[000, 010, 101, 110, 100]$ ，所以两行有 6 个公共的 1，即在一个编码组内，两行对应的数据块有 5 次公共的异或运算，行 6 对应数据块进行异或运算时直接基于行 5 的异或运算结果，就可以减少 5 次异或运算。按照依赖于已异或运算行的最终结果或者中间异或运算值，是两种不同的异或运算调度策略；且在不同情况下表现并不相同，需要对两种优化策略进行研究和比较。

3.4.5 编解码并发模块设计

编解码并发模块主要负责为基本编解码模块提供 Linux 线程池的并发计算功能。由于 RS 类编码方案在编码条带之间具有无关性，该并发执行功能能够根据 CPU 核心数以及光盘库存储系统负载情况，通过将编解码运算过程独立的多个条带同时执行实现；以充分利用 CPU 多核计算能力，从而有效加快编码或者解码的计算速度。

利用 Linux 线程池方法，可以有效地降低直接使用多线程并发技术在每次执行任务前创建线程、执行完成后销毁线程的 CPU 时间开销成本。例如，以对大小为 100GB 的蓝光光盘文件进行编码为例，如果采用约 32MB 的内存缓冲区，在四核并发执行的场景下，一个编码组即使在不分段执行的情况下，仍需要 12800 次线程创建操作以及 12800 次线程销毁操作；而使用 Linux 线程池方法会复用线程池内启动的线程，所以仅需要 4 次线程创建操作及 4 次线程销毁操作。图 3.6 是本系统所用 Linux 线程池的设计图。

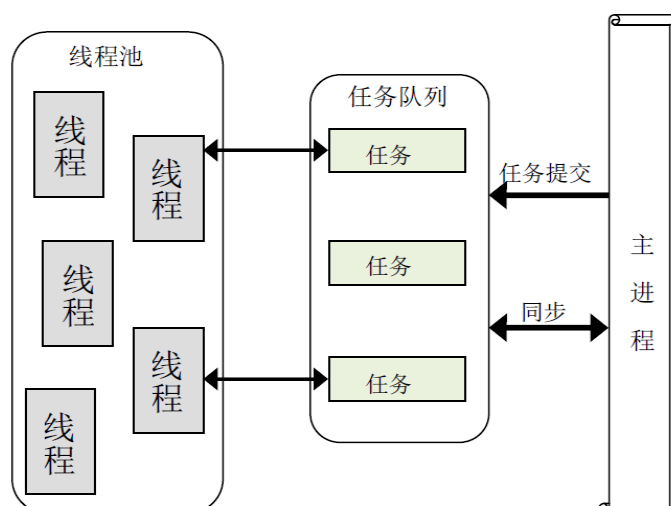


图 3.6 Linux 线程池设计图

该线程池的工作流程包括如下步骤：首先，创建指定数量线程，放入线程池，等待任务到达；然后，主进程提交任务后，激活线程池中某等待线程，由该线程从任务队列获取任务以执行；在执行批量任务时，主进程若需要等待该批任务完成，线程池会与主进程保持同步，在执行完成后向主进程发送完成信号。主进程在提交任务时，如果任务队列已满，则主进程进入等待状态。此外，主进程提交的任务包括待执行的函数以及该函数所需要的参数等信息。

3.5 本章小结

本章是对光盘库冗余纠错子系统的整体设计。首先，针对光盘库存储系统对可靠性的特有需求展开分析，清晰地获得冗余纠错系统所需完成的主要功能；然后，根据系统需求对冗余纠错系统与光盘库存储系统其它模块的接口进行设计，并对光

华 中 科 技 大 学 硕 士 学 位 论 文

盘库冗余纠错系统进行总体设计；最后，实现了系统功能模块划分，并将各个子模块所要完成的功能展开概要设计。通过本阶段对系统的总体设计，为冗余纠错系统关键算法的研究和具体实现奠定了基础。

4 关键模块和算法的研究及实现

本章将对系统涉及的关键算法展开研究，并将详细介绍系统若干主要模块的实现方法，包括基本编码模块实现、基本解码模块实现、异或运算调度算法研究和优化、线程池并发算法研究与实现等。

4.1 基本编码模块实现

4.1.1 柯西矩阵优化算法

启发式矩阵优化算法将采用一系列启发式矩阵初等变换运算实现，减少柯西矩阵对应的二进制分配矩阵中 1 的个数。

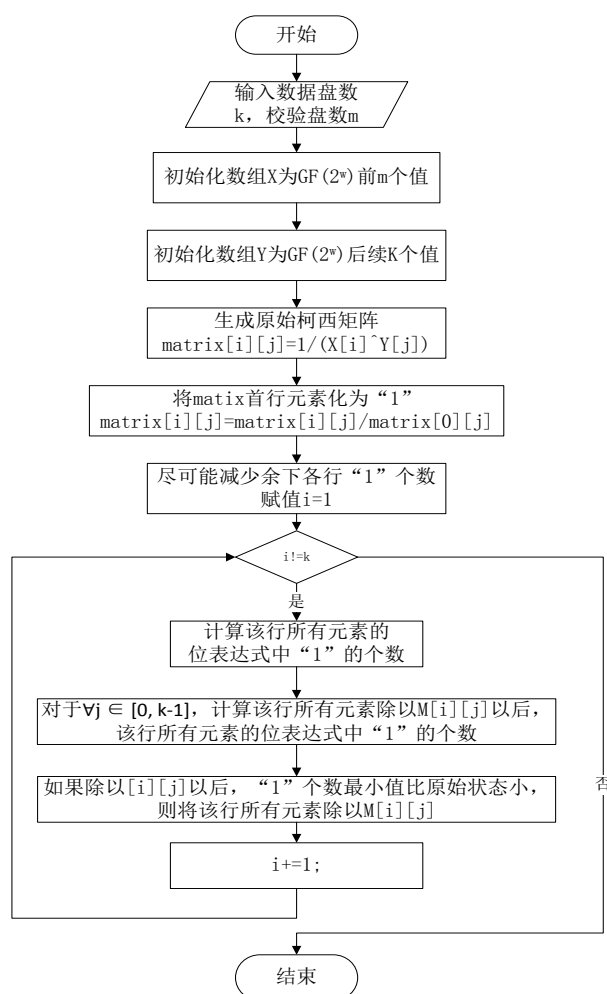


图 4.1 矩阵优化算法工作流图

图 4.1 是矩阵优化算法的工作流程图。该过程首先构建 CRS 编码所用 CM 矩阵，利用 $GF(2^w)$ 上前 m 个值为数组 X 的成员，后续 k 个值为数组 Y 的成员，直接生成 CM，相关加法及除法运算在 $GF(2^w)$ 中进行；然后，将矩阵每一列按域 $GF(2^w)$ 上除法运算除以该列首个元素，从而使得矩阵首行所有元素都转化为 1；最后，将遍历余下每一行，以尽可能减少该行所有元素位表达式中 1 个数，其优化基础是矩阵基本初等行变换，该类操作不会更改矩阵可逆性等性质。

4.1.2 异或命令生成实现

矩阵的位表达式方式被称作二进制分配矩阵(Binary Distribution Matrix，即 BDM)。对于 CRS 编码所采用的 BDM 元素中 1 的数目占比较低，在编解码过程中，为进行异或运算而遍历整个矩阵寻找元素 1，会进行很多不必要的遍历运算。例如，数据盘数目 k 取 10，校验盘数目 m 依次取 3、4、5，原始柯西 BDM 中 1 的个数占矩阵总元素的比例大多在 45%~55%，而优化 BDM 中 1 的个数占矩阵总元素的比例更低，大多在 25%~45%之间，具体如图 4.2 所示。所以，对于 CRS 编解码算法，通过遍历 BDM 寻找 1 的方式，大约产生 50%不必要的遍历运算，而对于优化矩阵不必要遍历运算比例甚至会达到 75%。

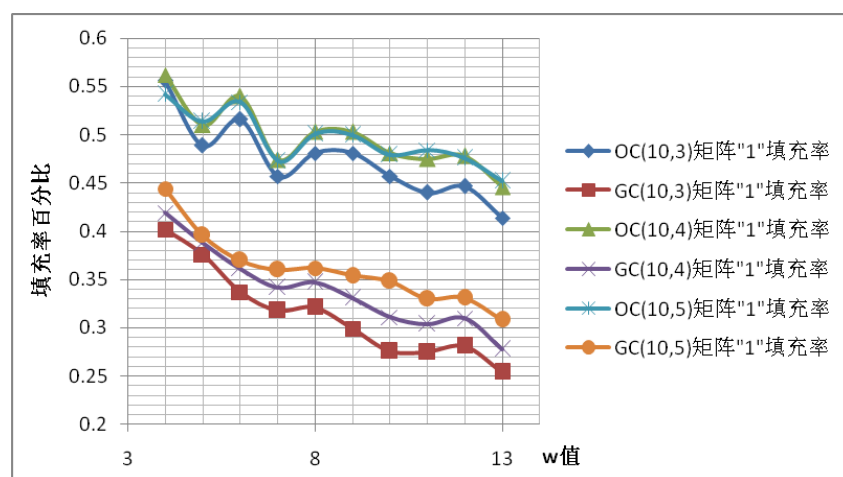


图 4.2 柯西 BDM 中元素 1 比率

通过将矩阵中的元素 1，转换成直接进行异或运算的命令，可以有效减少不必要的矩阵遍历运算；具体可以根据元素 1 在矩阵中的行列位置、 k 值、 m 值以及 w 值进行转换。可将异或运算命令设置为如下格式： $[S_d, S_b, D_d, D_b, Op]$ ，其中， S_d 为源设

备编号 (Source Device), S_b 为源设备位 (the Bit of Source Device), D_d 为目的设备编号 (Destination Device), D_b 为目的设备位 (the Bit of Destination Device), Op 为操作码 (Operation Code)。对于指令中的 Op , 如果等于 0 则表示将数据元素 (S_d, S_b) 复制到数据元素 (D_d, D_b); 如果等于 1 则表示将数据元素 (S_d, S_b) 与数据元素 (D_d, D_b) 进行异或运算, 并将异或运算的结果存入数据单元 (D_d, D_b) 原位置。

以编码算法为例, 具体将二进制分配矩阵中代表异或运算的元素 1 转换为异或运算命令的算法如下, 如果是二进制矩阵某行元素中首个 1, 则将 Op 赋值为 0, 否则将其赋值为 1; 对于数据盘个数等于 k , 校验盘个数等于 m , 盘内元素个数等于 w 的编码组, 位于矩阵第 i 行、第 j 列的元素 1 将 S_d 赋值为 j/w , S_b 赋值为 $j\%w$, D_d 赋值为 $k+i/w$, D_b 赋值为 $i\%w$; 其中, 对于 $k+m$ 个统一编号为 $0\sim k+m-1$ 的盘, 第 k 个盘表示首个校验盘。例如, $c_{0,0} = d_{0,1} \oplus d_{1,0} \oplus d_{2,3}$, 可以转换为 $[0,1,3,0,0]$, $[1,0,3,0,1]$ 以及 $[2,3,3,0,1]$ 三个异或运算指令。

4.1.3 光盘文件读写实现

光盘文件的读写通过文件读写操作列表实现。具体地, 根据每个文件的操作句柄及读写标志来定义类 Files; Files 包括两个成员变量: $flag$ 和 fp , fp 是文件操作句柄, $flag$ 是文件读写标志。对于 $flag$ 值, 0 代表该文件只可读, 用 rb 模式打开; 1 代表该文件只可写, 用 wb 模式打开; 2 代表该文件执行读和写/更新, 以 $rb+$ 方式打开; -1 代表不处理该文件。Files 类及文件操作列表 $vFiles$ 定义如下。

```
struct Files {  
    int flag; /* 0, 读; 1, 写; 2, 读和写; -1, 不处理 */  
    FILE *fp;  
};  
vector<Files> vFiles; /* 光盘文件读写列表 */
```

对于不分段进行编码方案或者分段编码方案参与编码过程的数据文件, $flag$ 赋值为 0, 而分段编码方案不参与编码过程的数据文件赋值为 -1; 而校验文件在不分段编码或者分段编码进行首段编码时, $flag$ 赋值为 1, 而分段编码方案进行非首段编码时 $flag$ 赋值为 2, 即此时用读写模式操作该光盘文件, 但打开时不会清除原有数据。

对于 flag 值等于 2，即以 rb+方式打开的光盘文件，其操作流程首先读取指定缓冲区 buffersize 大小的数据块，然后将光盘文件操作句柄 fp 后移 buffersize，待新编码结果得出后，再将该校验内容写入 fp 句柄当前位置。此外，当校验光盘文件比数据光盘文件大时，依然需要全部写入，这与条带化编码有关。校验光盘文件生成以后，需要将编码方案参数写入编码方案元数据文件，以备光盘文件重构使用；编码方案参数包括数据盘文件数 k 、校验盘文件数 m 、单文件条带内数据元素个数 w 、异或运算元素大小 packetsize 以及编码组文件列表等信息。

4.2 基本解码模块实现

4.2.1 逻辑地址映射算法

设置数组 $erased[0:k+m-1]$ 作为失效光盘位图，用以记录编码组内各个光盘文件的有效及失效情况，编码组内第 i 张光盘有效则将 $erased[i]$ 赋值为 0，失效则将 $erased[i]$ 赋值为 1；此外，如果失效光盘的数目 $numeraled$ 小于 m ，而解码重构只需要 k 张有效光盘，就会有 $m-numeraled$ 张光盘不会参与解码，将位图对应位赋值为 -1。

失效光盘位图 $erased[0:k+m-1]$ 中值 0、值 1 及值 -1 是交错分布的，即并没有连续排列在一起，这种情况并不便于矩阵运算的实现，所以，需要将离散的光盘物理地址，重新映射为连续的光盘逻辑地址，以便于解码矩阵生成异或计算命令。图 4.3 是编码组内参与解码光盘对应内存缓冲区映射关系示意图。该图中，数据盘有 5 张，校验盘有 3 张；其中，上面一行表示光盘失效情况，灰色表示发生故障；下面一行为物理地址映射后的逻辑地址，有效光盘和失效光盘各自连续排列。

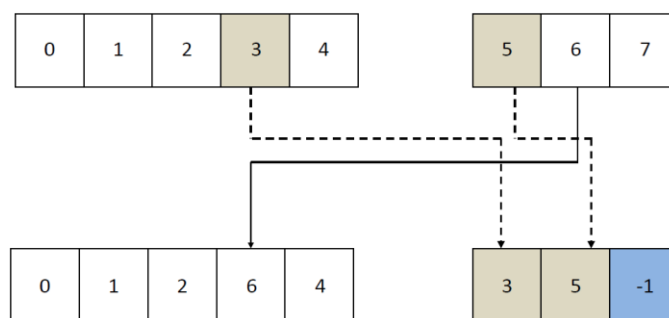


图 4.3 解码光盘物理编号与逻辑地址映射图

实现方式是遍历前 k 张数据盘，如果该盘失效，则将首个有效校验盘映射到该

位置，并将该失效数据盘映射到待解码盘首个空闲地址，依此直到检查完所有数据盘；然后将所有失效校验盘依此映射到失效数据盘后面的位置；最后，将不参与解码的光盘文件地址赋值为-1。

4.2.2 重构矩阵生成算法

解码算法所用重构矩阵依赖于编码矩阵。图 4.4 是解码矩阵生成工作图，整个工作流程分两部分，首先构建解码矩阵的数据盘重构超级行，再据此构建解码矩阵的校验盘重构超级行；该处超级行指柯西矩阵一行的二进制位表达式，超级列同此。

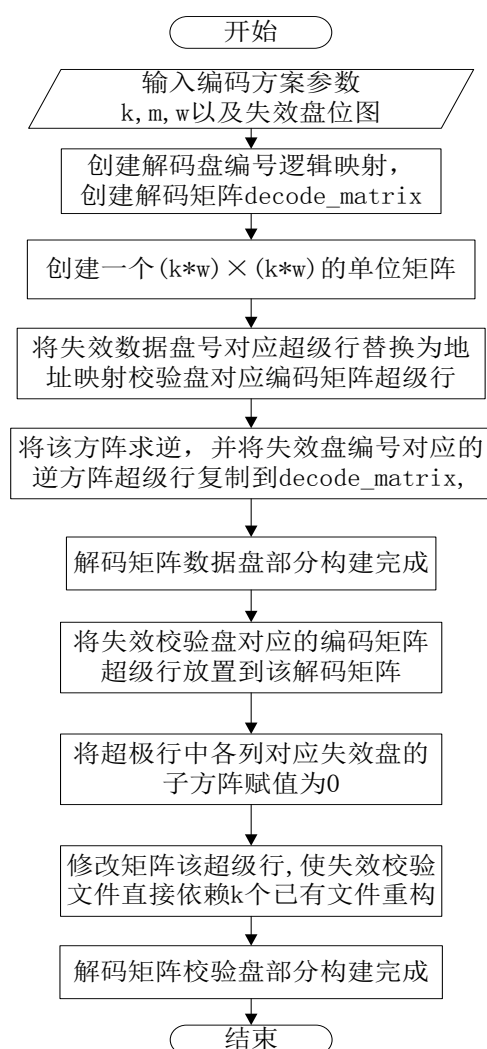


图 4.4 解码矩阵生成工作图

首先，计算数据光盘失效数量 df (Data Failures)，校验盘失效数量 cf (Code Failures)，依此创建 $k*w$ 列、 $(df+cf)*w$ 行的空解码矩阵 $decode_matrix$ 。然后，通过

编码矩阵转换成解码矩阵重构数据盘使用的超级行。最后，需要构建重构校验盘所用超级行，这主要基于编码矩阵原有超级行生成，难点在于原有超级行某些列对应的数据盘已经损坏，所以，需要更改该部分矩阵元素，以直接基于现有解码文件执行解码操作。实现的核心在于，该超级行对应失效数据盘的超级列，即该子方阵对应的数据块也需要重构；所以，可以直接基于该数据块的重构运算进行当前校验盘的重构。简单来说，如果编码超级行该子方阵内第 i 行第 j 列元素等于 1，那么需要将重构该数据文件超级行内第 j 行与重构该校验文件超级行内第 i 行进行异或运算。这样就实现直接利用重构失效数据盘的 k 个光盘文件，重构失效校验光盘文件。

4.2.3 光盘文件读写实现

解码模块读写光盘文件的方式与编码模块基本相同，同样使用类 Files 及文件操作列表 vFiles；不同之处在于，解码模块使用失效光盘位图 erased[0:k+m-1] 中值 0、值 1 及值 -1 来对文件列表中各项 flag 进行赋值。

有效光盘文件的数据内容直接读入内存缓冲，不进行地址映射；由上述地址映射模块将有效光盘文件对应的内存空间地址进行连续排列，从而达到简化解码过程的目的。当 erased[i] 为 1 时，在分段编码处理第一段时，将 flag 赋值为 1；其后各分段将 flag 赋值为 2，执行前述编码时相同的文件操作。

4.3 异或调度算法研究和优化

矩阵异或调度技术通过调整各校验元素的计算次序，从而实现矩阵公共元素的预先计算，这样就可以让后面运算充分使用前缀运算的结果，以此降低异或计算总次数。矩阵调度算法不但可优化编码过程，在重构过程中也具有显著优化效果。该算法由 J. S. Plank 提出，后在 Jerasure 等开源库及混合纠删码等研究中得到应用^[45]。

4.3.1 矩阵调度算法

点积 (Dot Product, 即 DP) 是 BDM 一行与元素乘以数据列计算得出的结果，如图 4.5 中的 $C_{0,0}$ ，图 4.5 是 $k=5$, $m=3$, $w=3$ 场景下编码示意图。如果采用直接利用矩阵各行元素编码的方式，由于 BDM 中有 63 个 1 且有 9 行，所以计算这 9 个点积元素需要 54 次异或运算，而理论最优值仅需 36 次异或计算。现举例用 BDM 矩阵

的行 5 和行 6 来计算 $C_{1,2}$ 和 $C_{2,0}$ 来说, 行 5 有 8 个 1, 行 6 有 9 个 1, 采用直接计算 $C_{1,2}$ 和 $C_{2,0}$ 的方法共执行 15 次异或计算。但是, 从图中可以看出 $C_{1,2}$ 和 $C_{2,0}$ 两行有 6 个公共元素 1; 因此, 如果现在先计算 $C_{1,2}$ 需执行 7 次异或运算, 然后再计算 $C_{2,0}$ 就可以用等式 $C_{2,0}=C_{2,2} \oplus D_{0,0} \oplus D_{0,2} \oplus D_{2,1} \oplus D_{3,2} \oplus D_{4,2}$ 实现。采用这种计算方式仅再需要 5 次异或运算即可得出 $C_{2,0}$, 从而使得计算 $C_{1,2}$ 和 $C_{2,0}$ 两个点积所需异或运算总数从 15 降低到 12。矩阵调度算法的核心正源于这种思想, 基于已有点积结果计算新点积, 从而比独立地计算各个点积需要更少异或运算总数。

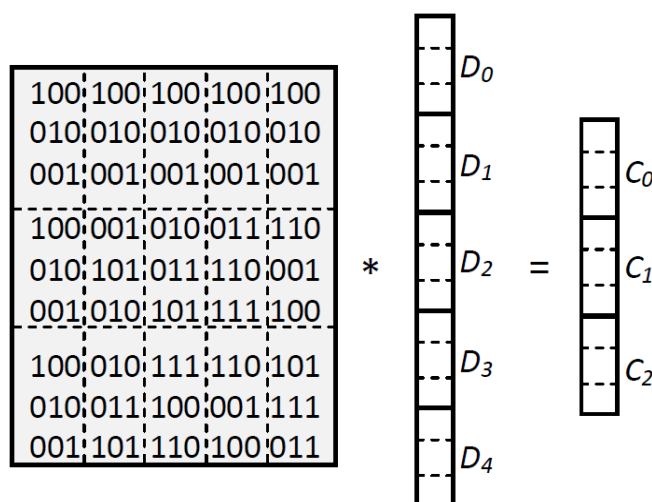


图 4.5 编码方案 (5,3,3) 编码过程示意图

矩阵调度算法相关假设和符号如下所述。

1. 将矩阵 M 乘以列数组 V 来计算得出点积列数组 U 。 M 、 V 以及 U 所有的元素都是二进制位, 即为 0 或者 1; 此外, 所有算术运算在域 $GF(2)$ 上执行。
2. 矩阵 M 大小为 m 行 k 列, 其中, 位于第 i 行第 j 列的数值记为 $M[i][j]$, M 的第 i 行记为 $M[i]$ 。数组 V 有 k 个元素, 分别记为 $V[0], \dots, V[k-1]$ 。数组 U 有 m 个元素, 分别记为 $U[0], \dots, U[m-1]$ 。即 $M_{m \times k} * V_{k \times 1} = U_{m \times 1}$ 。
3. $From[]$ 是一个包含 m 个整数的数组, 所有元素初始化为 -1。 $Ones[]$ 是一个由 m 个整数组成的数组, $Ones[i]$ 初始化为矩阵 M 中第 i 行中“1”的个数。 $ToDo$ 是一个由 0, 1, ..., $m-1$ 组成的整数集合, 代表需要计算的点的编号。 $Sum(i, j)$ 是一个由 k 个数值组成的位行数组, 其值为行 $M[i]$ 和行 $M[j]$ 两行之和。

设矩阵为 m 行, 则矩阵调度算法需要执行 m 步来完成, 每步执行如下四步运算。

1. 选择值 i , i 满足条件 $i \in \text{ToDo}$ 且 $\text{OnesMin}[i]$ 是数组 $\text{OnesMin}[]$ 的最小值。
2. 如果 $\text{From}[i]$ 等于 -1, 即没有可以依赖的异或运算组合, 则 $U[i]$ 直接等于使得 $M[i][j]=1$ 的所有 $V[j]$ 的异或结果。如果 $\text{From}[i]$ 不等于 -1, 即有可依赖的异或运算组合, 则 $U[i]$ 等于 $U[\text{From}[i]]$ 和所有使得 $M[i,j]+M[\text{From}[i],j]=1$ 的所有 $V[j]$ 的异或结果。
3. 从 ToDo 中移除 i 。对于所有 $j \in \text{ToDo}$, 计算行数组 $\text{Sum}(i, j)$ 即矩阵行 $M[i]$ 和行 $M[j]$ 的和, 并计算该数组元素 1 个数记为 x 。如果 x 小于 $\text{OnesMin}[j]$, 那么将 $\text{OnesMin}[j]$ 赋值为 x , 并将 $\text{From}[j]$ 赋值为 i 。

该计算方式为启发式算法, 第一步需要寻找 M 中元素 1 最少的行。第二步中异或条件 $M[i][j] + M[\text{From}[i]][j] = 1$ 分两种情况, 当 $M[i][j]=1$ 且 $M[\text{From}[i]][j]=0$, 表示行 i 中对应数据元素 $V[j]$ 需要参与 $U[i]$ 计算以加上数据元素 $V[j]$; 当 $M[i][j]=0$ 且 $M[\text{From}[i]][j]=1$, 表示行 $\text{From}[i]$ 中对应数据元素 $V[j]$ 需要参与 $U[i]$ 计算以消去数据元素 $V[j]$ 。第四步为计算点积 i 时, 判断其他行 j 是否可利用行 i 的异或运算结果。

相比直接独立计算该点积元素的方式, 矩阵调度算法采用基于其它点积元素的异或结果来计算当前点积元素的方式, 可以发挥高效的改进作用。例如在图 4.5 中, 如果使用该调度算法, 则可将独立计算点积元素所需 54 次异或运算, 减少至 49 次。

4.3.2 矩阵调度优化算法

假设现在先计算行 5 和行 6 的 6 个公共元素 1 并暂存入 $C_{1,2}$, 需要 5 次异或运算, 该中间结果记为 $C_{1,2}'$; 然后计算 $C_{2,0}$ 可用下面等式: $C_{2,0} = C_{1,2}' \oplus D_{0,0} \oplus D_{2,1} \oplus D_{4,2}$, 需要 3 次异或运算; 最后再计算 $C_{1,2} = C_{1,2}' \oplus D_{0,2} \oplus D_{3,2}$ 需要 2 次异或运算。这就使得计算 $C_{1,2}$ 和 $C_{2,0}$ 两个点积所需异或运算次数从 15 降低到 10, 即可减少 5 次异或运算, 而原有矩阵调度算法仅可减少 3 次异或运算; 从图中可知, 这是由于原有矩阵调度算法在减少异或运算的同时, 引入与 $D_{0,2}$ 和 $D_{3,2}$ 的异或运算, 从而降低了异或运算数优化能力。即采用基于最终编码结果的矩阵调度方式有其缺陷, 可以采用基于中间编码结果的方式进行优化。

矩阵调度优化算法是本系统研究内容的核心之一, 其核心思想是将矩阵具有最长公共元素 1 子序列的行进行两两匹配, 先计算出公共异或运算结果暂存入行 1, 然后行 2 依据行 1 的中间结果进行异或运算, 最后, 再得出行 1 的最终异或运算结果。

该算法的具体实现步骤如下所述。

1. 遍历优化矩阵，得出行 i 与后续每一行 j 的公共 1 长度，匹配行公共 1 个数列表 $llgroups_t[]$ ；同时，设置数组 $pro_ones_maxs_no_t[]$ 、 $pro_ones_maxs_t[]$ 以及变量 $row_ones_max_no_t$ ，其中， $pro_ones_maxs_no_t[i]$ 是与行 i 有最多公共 1 的行号， $pro_ones_maxs_t[i]$ 是行 i 与 $pro_ones_maxs_no_t[i]$ 的公共 1 个数， $row_ones_max_no_t$ 是公共最多的行号。
2. 根据 $row_ones_max_no_t$ ，将 $llgroups_t$ 中涉及 $row_ones_max_no_t$ 的匹配全部删除；同时，通过列表 $llgroups_t$ 更新 $pro_ones_maxs_t$ 和 $pro_ones_maxs_no_t$ 中涉及该行号的值，如果赋值为 -1，则表示已处理；并将该成对匹配行号放入 $qgroups_t$ 。重复该过程，直至 $llgroups_t$ 为空。
3. 遍历 $qgroups_t$ 生成调度命令，先将两行的公共运算结果存入行 i 对应的校验结果，行 j 直接基于该中间结果生成异或命令，然后行 i 生成其它异或命令，这些命令都依序进入异或调度命令队列，等待调度执行即可。
4. 遍历数组 $pro_ones_maxs_t$ ，将其中值为 0 的行 i ，即没有匹配的行，此时可根据矩阵元素直接生成异或运算命令，并进入异或调度命令队列。

表 4.1 矩阵调度优化算法计算步骤

计算顺序	待计算点积元素	依赖点积元素	异或运算次数
1	$C_{1,2}'$	计算公共“1”	5
2	$C_{2,0}$	$C_{1,2}'$	3
3	$C_{1,2}$	$C_{1,2}'$	2
4	$C_{1,1}'$	计算公共“1”	4
5	$C_{2,2}$	$C_{1,1}'$	3
6	$C_{1,1}$	$C_{1,1}'$	3
7	$C_{1,0}'$	计算公共“1”	3
8	$C_{2,1}$	$C_{1,0}'$	4
9	$C_{1,0}$	$C_{1,0}'$	3
10	$C_{0,1}$	直接计算	4
11	$C_{0,1}$	直接计算	4
12	$C_{0,2}$	直接计算	4

当依赖其它已经计算的点积元素中间结果而不是最终结果，来计算当前点积元

素时，矩阵调度优化算法可以发挥更高效改进作用。例如在图 4.5 中，如果使用矩阵调度优化算法，则可将独立计算点积元素所需 54 次异或运算，减少至 42 次异或运算，更加接近于理论最优值 36 次；该例使用矩阵调度优化算法时，编码过程会按照表 4.1 所示步骤计算目标点积。

对于 EVENODD 等已经非常接近理论最优性能 of 的算法，其可优化的空间已经非常有限，所以，矩阵调度优化算法并不能有效提升编码计算性能；但是对于 RS 类编码方案，矩阵调度优化算法不仅可以有效提升其编码运算速度，又可用于减少解码矩阵异或运算总数。

4.4 线程池并发算法研究和实现

Linux 线程池数据结构的定义如下所示。其中，变量 `mutex_threads` 与 `cond_threads` 是控制线程池内线程获取任务并执行的同步机制，而变量 `mutex_main` 和 `cond_main` 是控制并发的各任务与调用进程同步的机制，用以在执行完该批次任务后向调用进程发送完成信号，以使得主进程继续执行。

```
typedef void (*task_func)(void *arg);/* 任务函数原型 */
struct MyThreadPool {
    int threadnum;/*总线程数，不包括管理线程 */
    int tasknum_start;/* 任务队列首个任务编号 */
    int tasknum_end; /* 任务队列最后一个任务编号 */
    int tasknum_max;/* 任务队列容纳任务数 */
    pthread_mutex_t mutex_threads;/* 线程同步锁 */
    pthread_cond_t cond_threads;/* 线程同步的条件变量 */
    int tasknum_all;/* 调用进程提交任务数 */
    int tasknum_cur; /* 已执行所提交任务数 */
    pthread_mutex_t mutex_main;/* 调用进程同步锁 */
    pthread_cond_t cond_main;/* 调用进程同步的条件变量 */
    pthread_t *threads;/* 线程池内线程 */
    void **tasks_queue;/* 任务队列中传入函数的参数 */
    task_fun_ *tasks_func;/* 任务队列中传入的功能函数 */
};
```

华中科技大学硕士学位论文

文件读写任务传入参数的定义如下所示。通过 C++ 类初始化列表的形式将主进程中的参数传递给任务执行线程池。

```
struct RWArgs {  
    const vector<Files> &vFiles;  
    int k; int m; long buffersize; /* 缓冲区空间大小 */  
    char **data; /* 数据文件内存缓冲区 */  
    char **code; /* 校验文件内存缓冲区 */  
    RWArgs(const vector<Files> &vFiles_in, int k_in, int m_in, long  
    buffersize_in, char **data_in, char **code_in) : vFiles(vFiles_in), k(k_in),  
    m(m_in), buffersize(buffersize_in), data(data_in), code(code_in) { }  
};
```

编解码任务传入参数的定义如下所示。同样通过 C++ 类初始化列表的形式将主进程中的参数传递给任务执行线程池。其中，code_mid 是异或运算中间结果缓冲区，用以在分段进行编解码时计算出该段内异或运算的结果，然后将其与已生成的异或运算结果直接异或计算；这主要是异或调度算法矩阵各行计算时异或结果的依赖性所要求的。

```
struct CodeArgs {  
    int k; int m; int w; int **schedule; /* 异或运算命令 */  
    long packetsize; long codesize_sub_begin_t; long codesize_sub_end_t;  
    char **data; /* 数据文件内存缓冲区 */  
    char **code; /* 校验文件内存缓冲区 */  
    char **code_mid; /* 异或运算中间结果缓冲区 */  
    CodeArgs(int k_in = 0, int m_in = 0, int w_in = 0, int **schedule_in = NULL, char  
    **data_in = NULL, char **code_in = NULL, char **code_mid_in = NULL, long  
    packetsize_in = 0, long codesize_sub_begin_t_in = 0, long codesize_sub_end_t_in =  
    0) : k(k_in), m(m_in), w(w_in), schedule(schedule_in), data(data_in), code(code_in),  
    code_mid(code_mid_in), packetsize(packetsize_in), codesize_sub_begin_t  
    (codesize_sub_begin_t_in), codesize_sub_end_t(codesize_sub_end_t_in){ }  
};
```

分段编码场景下，并发编解码的内存布局如图 4.6 所示。该示意图中以两个并发线程为例，每个线程内部有多个条带，有一个缓冲区；线程内部是逐个条带进行编码或者解码的，中间结果暂存缓冲区，以达到调度优化算法的目的。

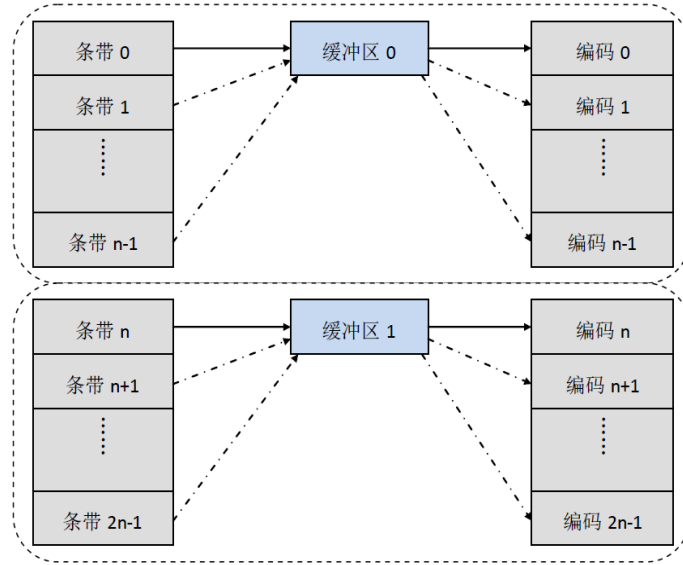


图 4.6 分段并发编解码示意图

4.5 纠错能力与时空开销分析

由于光盘编码基于文件进行，总体编码速度需要考虑从磁盘或 SSD 读取光盘文件的开销，以及将校验码写入校验文件的开销。设 S_{device} 为单个光盘文件的大小， R_{read} 为从磁盘读取文件数据到内存的速度， R_{write} 为从内存写入数据到磁盘文件的速度，则总体编码速度如公式 4.1 所示。

$$\begin{aligned}
 R_{\text{encoder}} &= \frac{k * S_{\text{device}}}{T_{\text{read}} + T_{\text{encoder}} + T_{\text{write}}} \\
 &= \frac{k * S_{\text{device}}}{\frac{k * S_{\text{device}}}{R_{\text{read}}} + \frac{k * S_{\text{device}}}{R_{\text{encoder}}} + \frac{m * S_{\text{device}}}{R_{\text{write}}}} \quad (4.1) \\
 &= \frac{k}{\frac{k}{R_{\text{read}}} + \frac{k}{R_{\text{encoder}}} + \frac{m}{R_{\text{write}}}}
 \end{aligned}$$

对于不分段编码的场景下，如果采用流水线技术，且将数据光盘文件与校验光盘文件放置在不同的磁盘或 SSD 上时，总体的编码速度如公式 4.2 所示。

$$R_{\text{encoder}} = \max \left\{ R_{\text{read}}, R_{\text{encoder}}, \frac{k}{m} * R_{\text{write}} \right\} \quad (4.2)$$

本研究将重点放在编解码方案的设计及其性能的优化上，并未对系统的整体吞吐率，即 IO 优化方面进行过多研究；这主要是由于在分段编解码场景，外存读写以

及编解码过程多有相互依赖和交叉，难以用流水化实现。性能优化的意义在于，一方面可降低系统的总体负载，另一方面并发执行提升速度，以满足未来 SSD 等替换磁盘作为光盘文件缓冲区场景下的性能提升需求。同时，由于冗余纠错系统采用 MDS 类型的 CRS 编码，所以在添加 m 张校验光盘下，可以容编码组内任意 m 张光盘失效。

4.6 本章小结

本章主要研究了冗余纠错系统涉及的关键算法，并详细介绍了系统若干主要模块的具体实现，包括基本编码模块实现、基本解码模块实现、异或运算调度算法研究和优化、线程池并发算法研究与实现以及纠错能力和时空开销分析等。

5 系统测试

5.1 测试环境

测试使用一台装有 Linux 操作系统的服务器，该主机装有 g++ 等软件，表 5.1 是系统测试软件环境。

表 5.1 系统测试软件环境

软件	版本
操作系统	Ubuntu 12.04.5 LTS
内核版本	3.13.0-32-generic x86_64
g++	4.6.3-1ubuntu5

硬件上，主要通过单主机进行测试；CPU 为 Q9400 型因特尔酷睿多核处理器，主频 2.66GHz，核心数为 4，线程数为 4，数据宽度等于 64 位，一级 Cache 容量等于 128KB，二级 Cache 为 6MB 共享缓存；内存大小为 4GB，类型 DDRII，频率为 800MHz；主机采用两块硬盘，型号皆为西部数据 WD10EARS，接口为 SATAII，转速 5400RPM，容量各为 1000GB，dd 命令测试读取速度大约等于 87MB/S，写入速度大约等于 67MB/S。

5.2 系统功能测试

系统测试主要针对具体的编解码模块以及编解码各阶段相关优化算法展开。本系统采用 C/C++ 编程语言实现，可以在大多数 UNIX 平台上执行，如 Ubuntu 等。该小节主要对冗余纠错系统基本功能进行测试。

首先，利用测试用数据文件生成模块，自动生成 k 个由随机字符组成的数据光盘文件；然后，初始化纠错码所需的域结构，如构建生成矩阵等；第三步，利用基本编码模块，通过依次调用编解码分段模块、编解码优化模块以及编解码并发模块等，将 k 个数据光盘文件编码获得 m 个校验光盘文件，校验文件和数据文件分别存储在专用磁盘；第四步，备份该编码组内所有光盘文件，并破坏一定数目的光盘文件，包括数据文件以及校验文件；第五步，利用基本解码模块，通过依次调用编解

码分段模块、编解码优化模块以及编解码并发模块等，从剩余的有效光盘文件中重构失效的光盘文件，包括数据文件以及校验文件；第六步，将恢复的光盘文件与初始备份下来的光盘文件进行比对，具体利用 Linux 下 diff 命令实现，以确认解码算法的正确性。

经测试，可以正常通过编码算法产生校验光盘文件，如图 5.1 调度优化编码结果图所示，该测试例子取 $k=20$ ， $m=6$ ， $w=5$ ；并可以通过解码算法重构光盘文件，包括数据文件以及校验文件，且经过 diff 命令比对，确认光盘文件重构无误，如图 5.2 所示。



图 5.1 $k=20$ ， $m=6$ 编码结果图

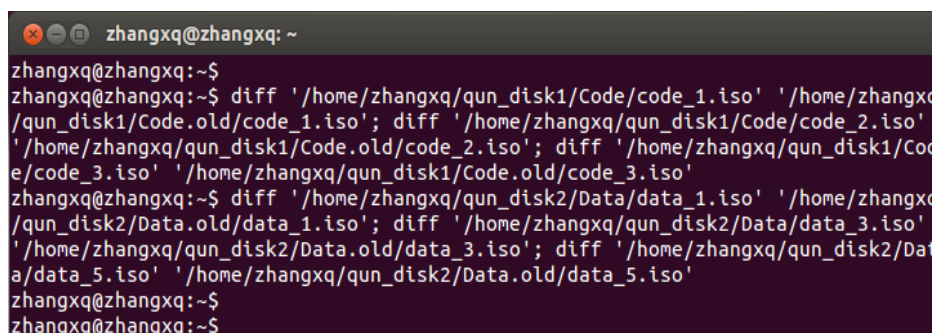


图 5.2 $k=20$ ， $m=6$ 重构文件比对图

5.3 关键算法优化测试

本小节主要对冗余纠错系统不同矩阵调度算法的异或次数进行测试。

1. 不同 w 值异或次数测试

图 5.3 是不同 w 值编码异或次数测试图，其中 k 等于 10， m 依次为 3、4、5，并分别用原始柯西矩阵以及优化柯西矩阵进行测试，而 w 的值域为 $[4, 13]$ ，其中 4 为 $GF(2^w)$ 域上最小合法值。通过纵向比较可以看出，不管是原始柯西矩阵还是优化

柯西矩阵，编码字异或运算次数都会随着校验盘数目的增长而增长；通过横向曲线可以看出，随着 w 值的增大，不管是原始柯西矩阵还是优化柯西矩阵，编码字异或运算次数都发生显著的增长；所以，首个合法的 w 值，即对于组合 (k, m) 的最小合法 w 值的编码方案比 w 值较大方案有更小的数据字异或运算次数。数据字异或运算次数是通过计算平均每个数据字节异或运算次数得出的，其大小可以直接反应编码速度的快慢。

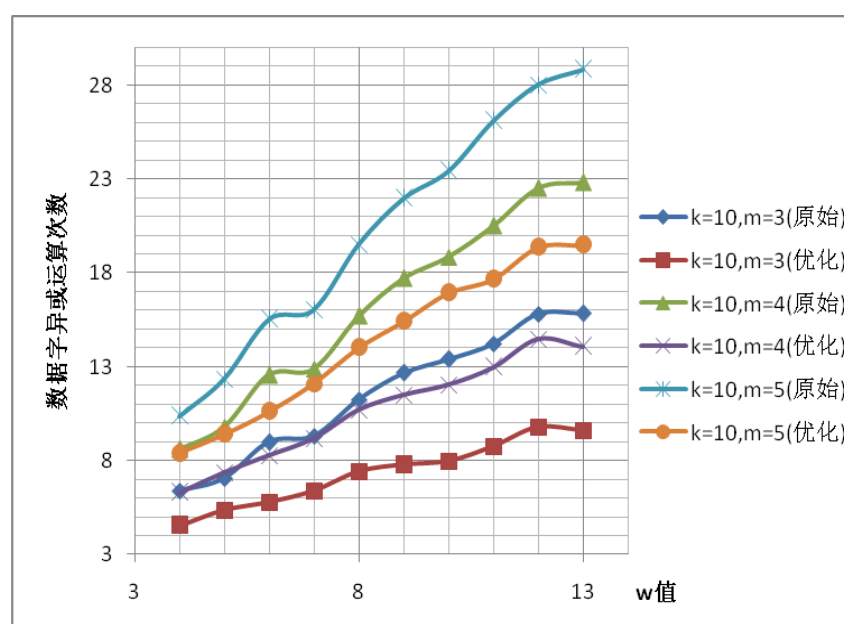


图 5.3 不同 w 值编码异或次数测试图

2. 调度优化算法编码测试

图 5.4 是段数为 1 数据字异或运算次数统计图，即对编码生成矩阵整体处理时，各种算法的数据字异或次数。其中，柱形图是不同 (k, m) 组合下，采用不同矩阵优化及调度优化算法情况的数据字异或运算次数，其中 (k, m) 组合包括 $(10, 3)$ 、 $(10, 4)$ 、 $(10, 5)$ 、 $(20, 6)$ 、 $(20, 8)$ 、 $(20, 10)$ 六种数据盘数目及校验盘数目组合方式；而各种组合情况下，测试的算法包括标准柯西矩阵、优化柯西矩阵算法、对优化柯西矩阵调度算法、对优化柯西矩阵优化调度算法以及理论最优值等五种情况，使用主纵坐标轴。折线图代表矩阵调度算法和矩阵优化调度算法，相比直接使用优化柯西矩阵编码的数据字异或运算数减少比率，使用次纵坐标轴。

通过柱状图分析可得出，由于优化矩阵算法是直接进行异或运算，故说明其可

以显著减少矩阵元素 1 总数，即生成矩阵优化算法可以显著地减少矩阵中 1 个数，从而为加快编码速度提供可能；基于优化矩阵的调度算法通过利用相同异或运算结果，在一定程度上减少编码数据字异或运算数；而对优化矩阵采用优化调度算法，则可以比原始调度算法更加有效地减少数据字异或运算数。根据图中曲线可知，在系列测试场景下，基于优化矩阵的调度算法通过调度矩阵异或运算顺序而减少运算的比率范围为 7.7%~15.8%，而基于优化矩阵的优化调度算法减少异或运算的比率范围为 23.2%~26.8%，大约为原始调度算法减少异或运算数能力的 2 倍，甚至在组合(10, 3)场景下达到 3 倍，而最差优化情况也达到 1.6 倍。

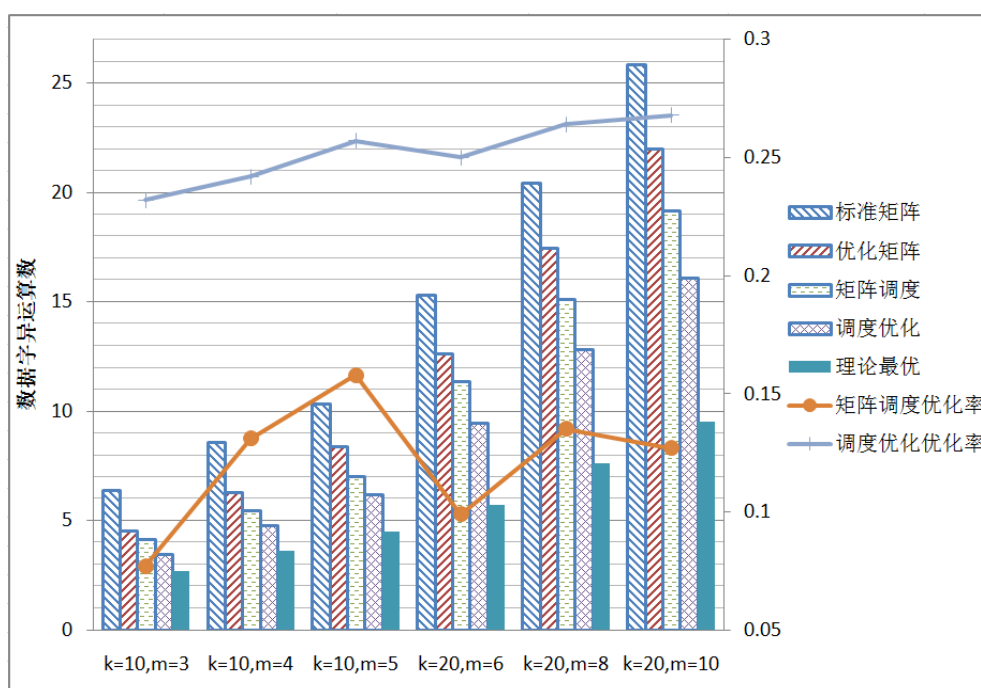


图 5.4 段数为 1 数据字异或运算次数统计图

图 5.5 是段数为 2 数据字异或运算数统计图，参数与图 5.4 基本相同，不同在于将矩阵分两段处理；所以，理论最优值、标准柯西矩阵、优化柯西矩阵算法等三种情况不变，而优化柯西矩阵调度算法及优化柯西矩阵优化调度算法由于调度规模变小，数据字异或计算次数表现有所不同。根据图中曲线可知，在分两段调度的场景下，对优化矩阵采用原始调度算法减少异或运算次数的比率范围为 11.0%~23.3%，而对优化矩阵采用优化调度算法减少运算的比率范围为 22.1%~27.6%；基于优化矩阵的调度算法优化能力有所提升，这主要得益于分段处理后调度算法中使得优化变

弱的因子有所减轻。总体来看，大多数编码情况下基于优化矩阵的优化调度算法仍然比原始调度算法减少异或运算的能力要强很多。

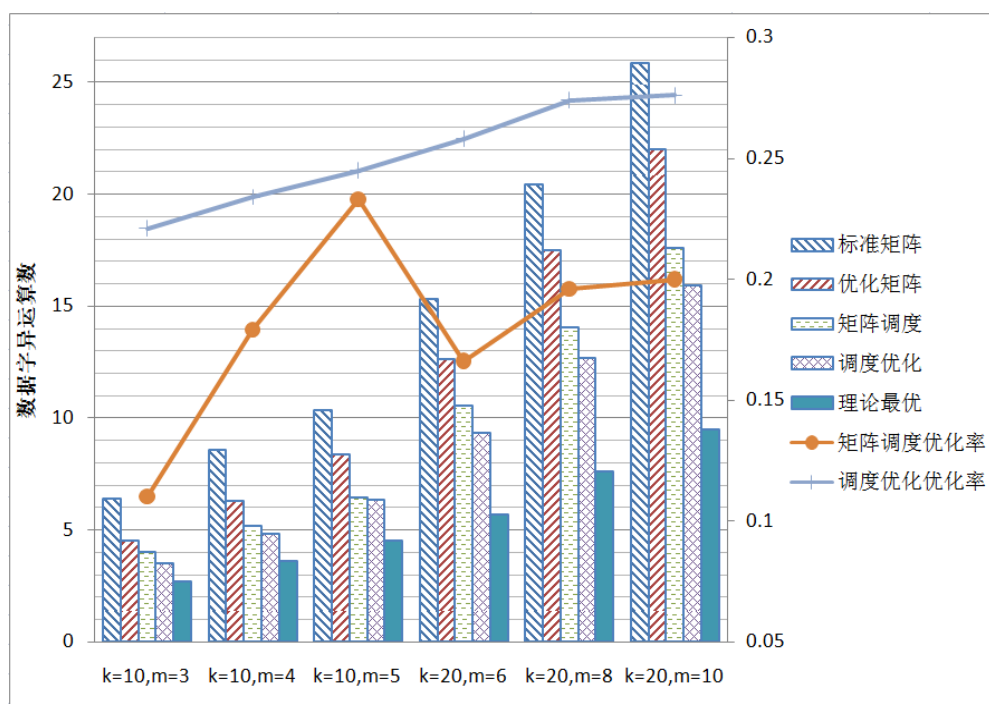


图 5.5 段数为 2 数据字异或运算数统计图

3. 调度优化算法解码测试

在单盘损坏的情况下，重构算法采用编码组内首个冗余校验文件进行数据恢复，可以达到理论最优值；所以该部分仅对失效盘数目大于 1 的情况进行测试。参考 Windows Azure 基于 RS 编码的冗余率在 1.3~1.5 之间，由于本算法基于标准 RS 编码，并未做校验盘数量宽松处理，所以，该处仅对冗余率 1.3 和 1.4 两个值进行测试。

图 5.6 是段数为 1 编码字异或运算数统计图，即对解码二进制分配矩阵整体处理时，两种调度算法情况下，编码字异或次数的。柱形图是不同 (k', m') 组合下，采用矩阵调度及矩阵调度优化算法情况下，编码字异或运算次数，使用主纵坐标轴；其中， k' 是指失效的数据光盘数， m' 是失效的校验光盘数； (k', m') 组合在 $k=20, m=6$ 场景下包括 (3,0)、(6,0)、(3,3) 三种情况，在 $k=20, m=8$ 场景下包括 (4,0)、(8,0)、(4,4) 三种情况。折线图代表矩阵调度算法和矩阵优化调度算法，相比直接使用解码矩阵重构的编码字异或运算数减少比率，使用次纵坐标轴。该处编码字是指重构失效文件

的字节所需异或运算次数。

通过柱状图分析可以得出，矩阵调度算法通过利用公共异或运算结果，在一定程度上减少编码字异或运算次数；而矩阵优化调度算法，则可以比简单的调度算法更加有效地减少编码字异或运算数。根据图中曲线可知，在系列测试场景下，矩阵调度算法通过调度矩阵异或运算顺序而减少运算的比率范围为 13.3%~20.7%，而矩阵优化调度算法减少异或运算的比率范围为 27.3%~29.7%，大约为原始矩阵调度算法减少异或运算次数能力的 1.7 倍，最差优化情况也达到 1.43 倍。

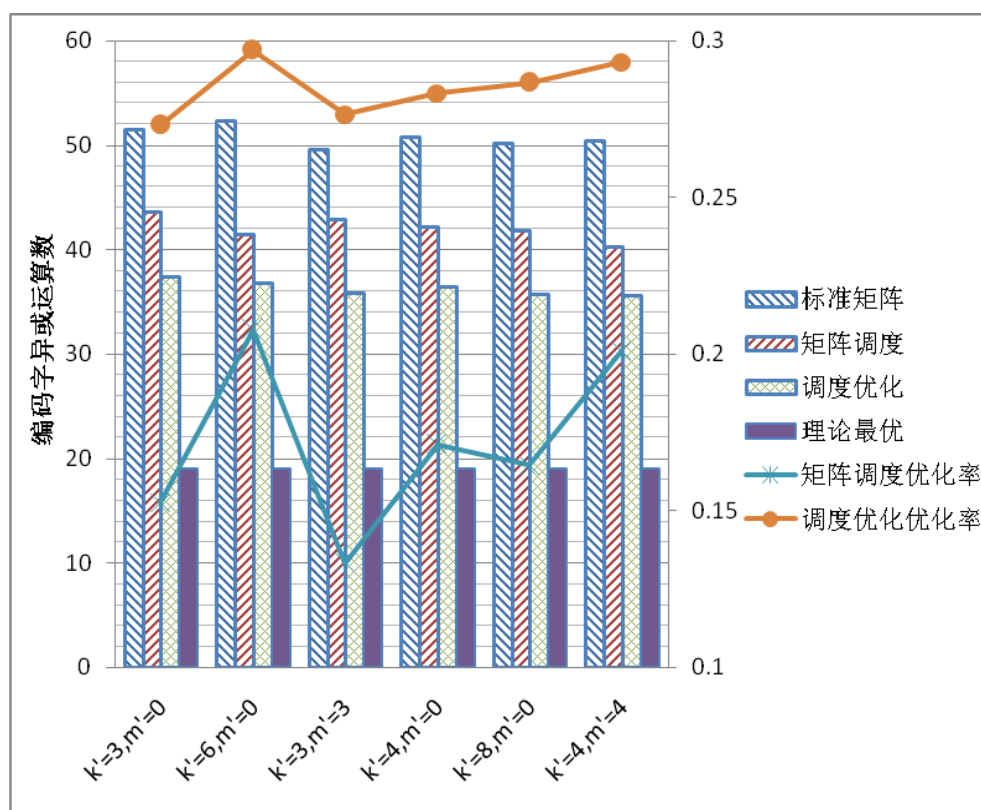


图 5.6 段数为 1 编码字异或运算次数统计图

图 5.7 是段数为 2 编码字异或运算次数统计图，参数与图 5.6 基本相同，不同在于将矩阵分两段处理。根据图中曲线可知，在分两段调度的场景下，矩阵调度算法减少异或运算的比率范围为 18.0%~26.6%，而矩阵优化调度算法减少运算的比率范围为 27.0%~30.5%；矩阵调度算法优化能力有所提升，同样得益于分段处理后调度算法中使得优化变弱的因子有所减轻。总体来看，大多数解码情况下，矩阵优化调度算法仍然比矩阵调度算法减少异或运算的能力要强。

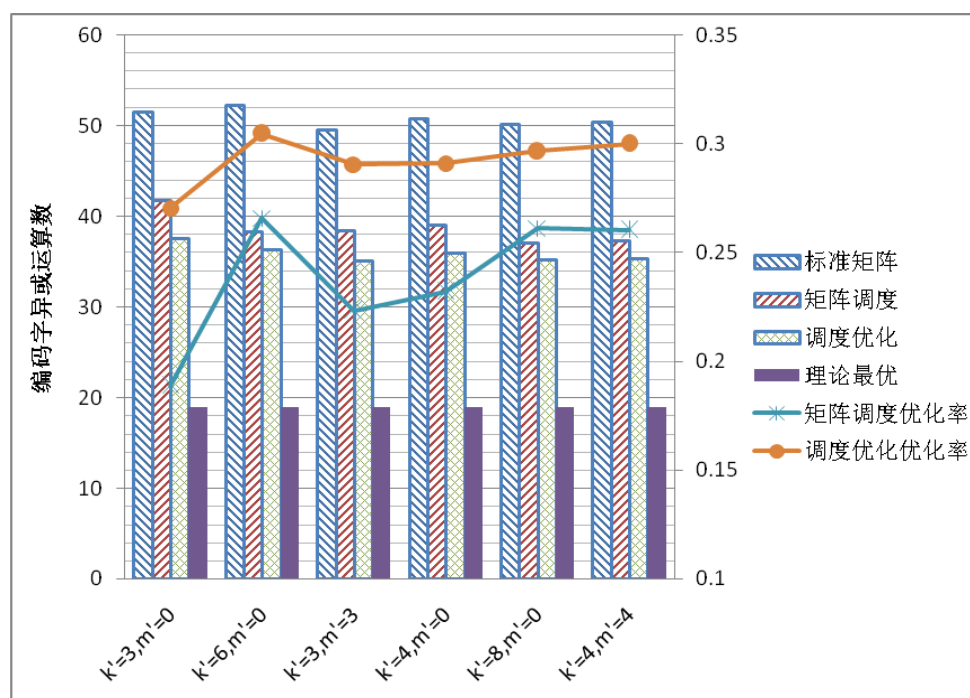


图 5.7 段数为 2 编码字异或运算次数统计图

5.4 系统性能优化测试

本小节主要对冗余纠错系统的编解码性能进行测试。

1. 不同 m 值、packet size 值编码性能测试

图 5.8 是标准 CRS 编码不同 w 值性能图，其中 $k=10$ ， $m=3$ ， w 值域为 $[4, 13]$ 。图 5.9 是异或调度优化 CRS 编码不同 w 值性能图，其中 $k=10$ ， $m=3$ ， w 值域为 $[4, 13]$ 。两幅图横坐标都是 w 值，代表域 $GF(2^w)$ 的选择；纵坐标都是编码速度，单位 MB/S。由曲线图可以看出不论是标准 CRS 编码，还是异或调度优化 CRS 编码，在异或运算元素不同 packet size 值下，都是随着 w 值的增大，异或运算的编码速度都有着明显的降低；这是由于随着 w 值增大，数据字平均异或运算数明显增大，使得编码进行更多地异或运算而引起的。

图 5.10 是不同 packet size 值编码性能图，其中 k 与 m 分别取 $(10,3)$ 、 $(20,6)$ ；且对两组 (k,m) 值分别用标准 CRS 编码和异或调度优化 CRS 编码进行测试。由曲线图可以看出，在四种编码场景下，随着 packet size 值的增大，编码速度都有着明显的加快；然后，达到某一峰值后，编码速度不再明显增长，甚至会随着 packet size 值

的增大而出现小幅降低的情况。出现这种情况，首先是因为随着 `packetsize` 值增大，单个条带处理的数据量增大，这样不仅可以减少异或运算命令遍历次数，减少异或数据访问次数，更可以充分利用高速缓存以有效地加快编码速度；而随着 `packetsize` 过度增长，使得高速缓存内容替换变得频繁，不能充分利用编码条带内部局部性，虽然异或运算命令遍历次数依然减少，但仍使得编码速度有所下降。

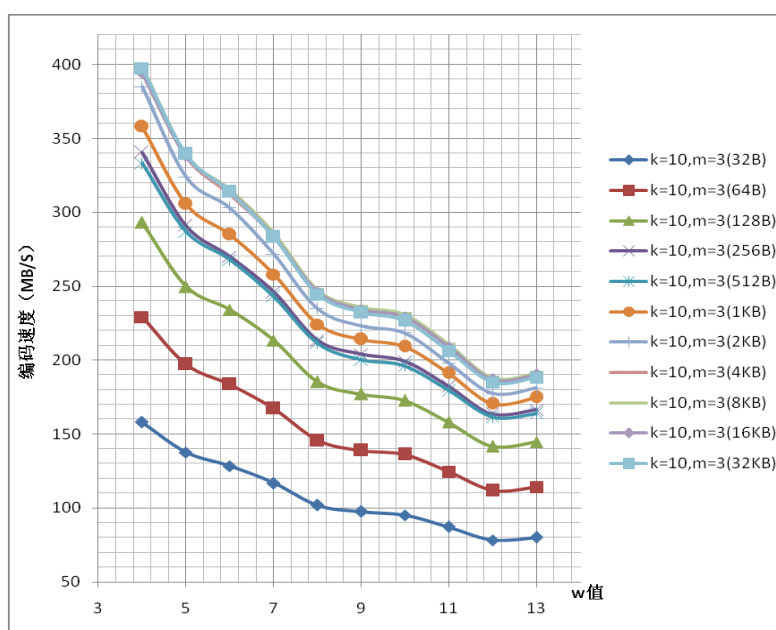


图 5.8 $k=10, m=3$ 标准 CRS 编码不同 w 值性能图

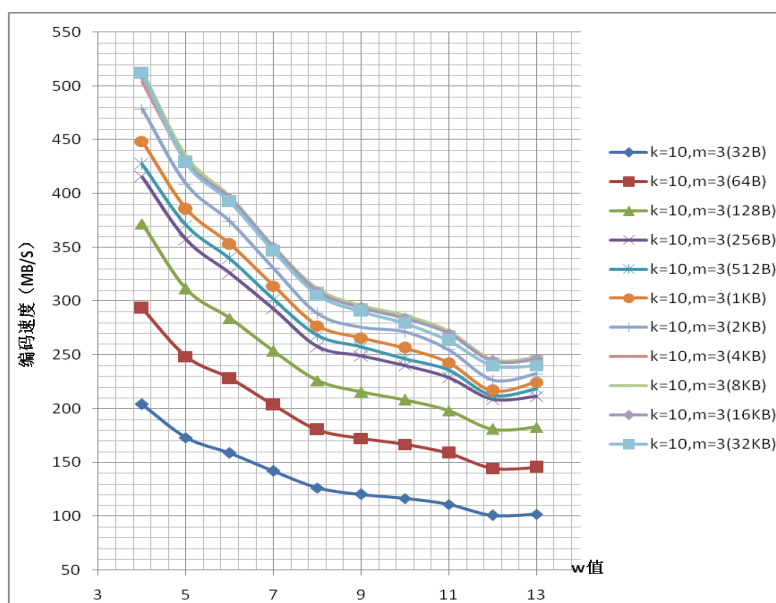


图 5.9 $k=10, m=3$ 优化 CRS 编码不同 w 值性能图

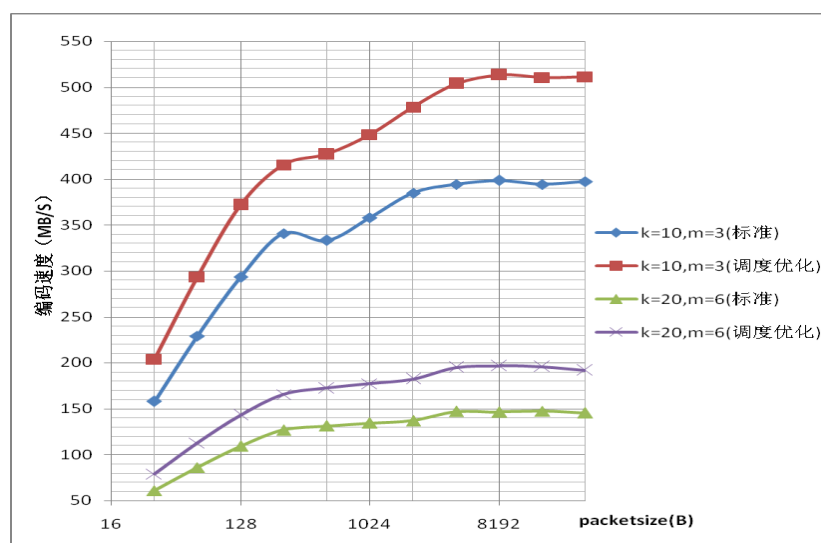


图 5.10 不同 packetsize 值编码性能图

2. 调度优化算法编码性能测试

图 5.11 是 $k=20$, $m=6$ 及 $m=8$, w 取最优的编码场景下, 各种方案的单线程性能比较图。其中, 分段情况为将矩阵分成两段; 由于标准矩阵编码未采用任何优化, 分段编码处理对其几乎无影响; 而矩阵调度与调度优化算法则略有差异, 基本上与调度优化算法测试结果相符合。在该测试例子两种码率情况下, 相比标准矩阵算法, 矩阵调度算法性能分别提升 11.2% 和 14.9%, 而调度优化算法性能分别提升 32.9% 和 35.5%, 其提升能力均超过原有矩阵调度算法性能改进能力的两倍。

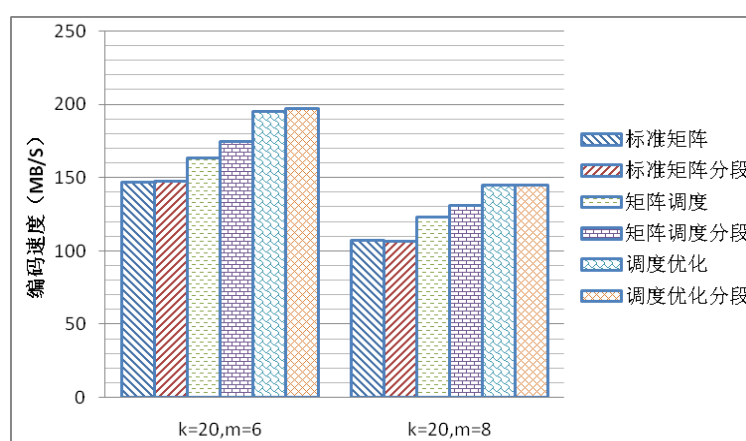


图 5.11 最优 w 下编码性能图

图 5.12 是对调度优化算法多线程编码性能的测试, 基本参数与图 5.11 测试相同。并发度为 2 编码方案性能是并发度为 1 方案的 2.0 倍和 1.98 倍, 并发度为 4 编码方

案性能是并发度为 1 方案的 3.85 倍和 3.87 倍。对于 $k=20$, $m=6$ 不分段的情况, 按光盘文件大小 100GB 编码计算, 标准矩阵需要 11.63 分钟, 矩阵调度需要 10.43 分钟, 调度优化需要 8.70 分钟; 而调度优化算法按并发度为 4 执行, 仅需要 2.26 分钟。分两段单线程编码场景的 IO 开销为 42 分钟, 并发度为 2 的 IO 开销为 32 分钟; 并发度为 2 方案开销降低原因在于执行第二段编码时, 并发读取数据文件和校验文件。

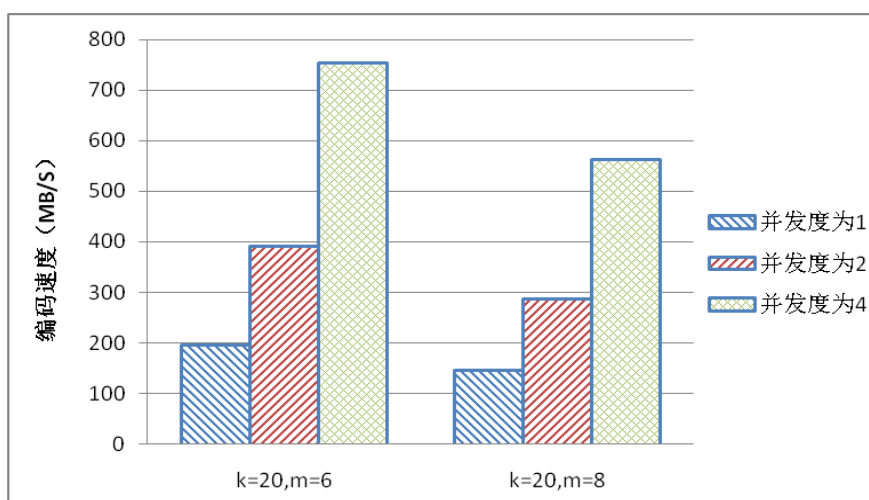


图 5.12 调度优化算法并发编码性能图

3. 调度优化算法解码性能测试

图 5.13 是 $k=20$, $m=6$, w 取最优的解码场景下, 各种解码方案的单线程解码性能比较图; 其中, 各种分段方案为将矩阵分割成两段处理。 k' 表示损坏的数据光盘数, m' 表示损坏的校验光盘数; 在三种光盘失效场景下, 相比标准矩阵算法, 矩阵调度算法性能分别提升 17.1%、25.5% 和 21.3%, 而调度优化算法性能分别提升 36.5%、40.9% 和 39.6%, 其提升能力均超过原有矩阵调度算法提升能力的 1.6 倍。此外, 解码速度比编码速度低很多, 这是因为解码性能是按照重构的光盘文件总大小除以解码时间所得, 而编码性能是按照编码的数据文件总大小除以编码时间所得。

图 5.14 是 $k=20$, $m=6$, w 取最优的不分段场景下, 对调度优化算法多线程解码性能的测试; 其中, k' 和 m' 的含义同上; 在三种光盘失效场景下, 并发度为 2 性能依次是不并发的 1.98 倍、1.98 倍和 1.97 倍, 并发度为 4 性能依次是不并发的 3.84 倍、3.89 倍和 3.86 倍。对于 $k=20$, $m=6$ 不分段解码的情况, 若失效盘数 $m'=6$, 按光盘文件大小 100GB 解码计算, 直接计算需 47.48 分钟, 矩阵调度需 37.80 分钟, 调度

优化需 33.15 分钟；如果调度优化算法按并发度为 4 执行，则仅需 8.5 分钟。在 $k'=3$, $m'=3$, 不分段解码场景下，不并发执行的 IO 开销为 75 分钟，并发度为 2 的 IO 开销为 60 分钟；并发度为 2 方案开销降低是因为并发读写数据文件和校验文件。

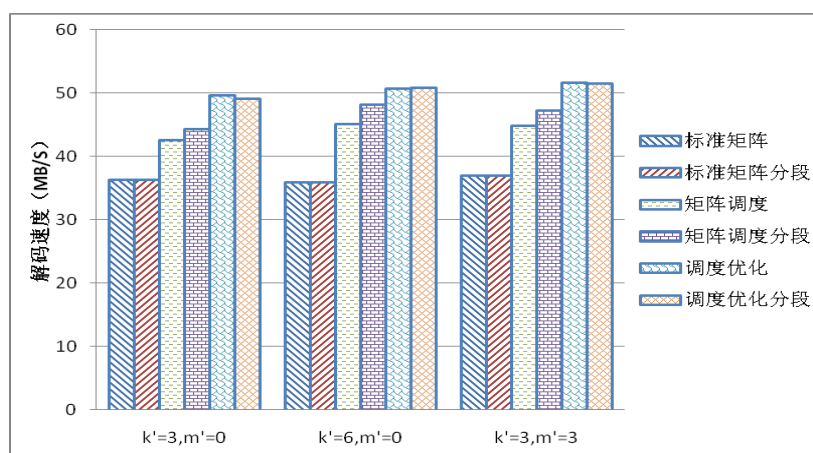


图 5.13 最优 w 下解码性能图

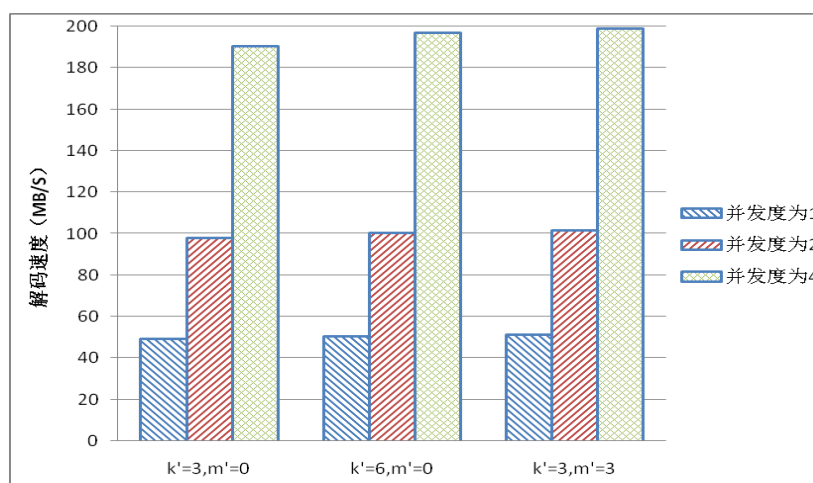


图 5.14 调度优化算法并发解码性能图

5.5 测试分析及结论

经过以上一系列的测试，可以看出光盘库冗余纠错系统的基本功能已实现，能够实现光盘库存储系统对多张光盘纠错能力的需求；并充分证明本文设计的新型调度优化算法可以有效提升编解码算法在各种场景下的性能，同时，可以有效缩短数据光盘文件编码、失效光盘重建过程周期，亦即提升了整个系统的可靠性和可用性。最后，测试说明 Linux 线程池技术可以有效加快编解码速度。

6 总结与展望

6.1 全文总结

对于企业而言，随着信息化、无纸化办公的快速变革，企业数据量每天都在急剧膨胀，数据的类型也各不相同；这些数据是企业赖以生存和发展重要基础，某些重要数据的损坏丢失将给企业带来无可挽回的毁灭性打击，因此需要安全、完整的保存。光盘库存储系统作为归档系统类型的一种，其冗余纠错子系统有其不同的特点与纠错能力需求。归档类型的存储系统，首要满足的是高容错能力以及高存储利用率，即需要较多校验光盘以容较多的光盘故障；而由于归档系统成本的限制，具有高容错能力、低码率的 MDS 编码成为不二选择。本文就是从此需求出发，对光盘库多盘片失效的数据恢复技术展开研究，并设计了该光盘库冗余纠错子系统，所做的主要工作总结如下：

1. 通过分析光盘库冗余纠错系统的需求以及各种冗余纠错技术的研究现状，提出了采用 CRS 编码作为该纠错系统的基本编码算法。
2. 分析了光盘库冗余纠错系统与光盘库存储系统其它各个模块的接口，设计了光盘库冗余纠错系统的总体架构，并按照功能进行模块化设计，详细介绍了各模块的功能与实现，并将编解码共同使用的功能提取为单独模块，便于系统实现。
3. 设计了适合该纠错系统的 Linux 线程池方案，为未来编解码性能提升的需求提供了充足的保障。
4. 根据光盘库存储系统独有特点，提出了分段进行编解码的设计方案，并对其性能和开销进行了评估、测试。
5. 重点对 CRS 编码的矩阵调度算法进行优化，提出了调度优化算法，从根本上减少了编解码异或运算的总数，提升算法性能的同时，降低了系统的总体负载。
6. 通过冗余纠错系统进行光盘文件分段编码，以及分段解码测试显示，可有效地应对光盘库多盘片失效的情况。测试新型矩阵调度优化算法显示，该算法可有效减少异或运算总数，优化能力在编码时约为原有算法的 2 倍，在解码时约为 1.7 倍。

6.2 不足与展望

虽然本文设计实现的光盘库冗余纠错系统能够基本完成系统实际需求，但由于时间和技术的制约，仍存在些许需改进的地方：

1. 对于磁盘文件的读写操作，该纠错系统采用串行方式实现，仅对数据盘和校验盘进行了并行读写，受限于分段编解码的光盘文件读写过程交叉属性，未能充分地将编解码过程实现流水化处理；未来可以借助线程池技术，最大化地挖掘光盘文件读写操作的并行度，从而加快编解码操作。

2. 对于光盘库存储系统可靠性的评估还不够充分，目前主要借鉴 Windows Azure 基于 RS 编码改进型存储所用的冗余率，未来需要考虑构建评估光盘存储介质系统可靠性的模型，以为具体需求提供具有量化可靠性的冗余率方案^[46]。

3. 在分段编解码模式下，校验光盘文件的读写操作频率要比数据光盘文件高很多，所以对应磁盘损坏的概率也要高很多；未来可利用 SSD 中提出的读写均衡算法，使得各个磁盘损耗基本相等；或者直接采用极度不均衡策略，将磁盘损耗集中在部分盘上，从而方便磁盘更换，增加整个系统的可靠性。

4. 在数据重构时，该系统采用所有数据统一重构的解码模式，并没有考虑用户对于数据请求长时间等待的问题；未来可以调整解码算法执行过程，优先恢复用户请求的局部数据，以最小化用户等待时间，从而提升用户使用体验。

致 谢

时光如白驹过隙，整整三年的硕士研究生学习经历行将完结，在这个过程中，我不仅仅获得了比较丰富的专业知识，自身素质的成长也是非常明显的，思维思考的方式更加开阔，科研做事的风格更加严谨。而对我更加弥足珍贵的是学校浓厚的学术氛围，实验室共同学习研究的科研环境，以及跟同学们浓厚的友谊。在行将毕业的季节，我对所有关心帮助过我的父母、导师以及同学表示由衷的感谢。

首先，我要对我的父母表示由衷的感谢，是你们在我二十多年成长和学习的过程中，给予我无私的关心爱护；尤其是在学习生活中我取得好成绩时祝福我，同时又勉励我；当我遇到挫折时，总是予以我鼓励和开导。你们一直以来无私的支持和爱，是我前进的永恒动力；在此，我要对你们多年的付出表示由衷的感恩。

然后，我要对导师林安老师表示衷心的感谢。林老师不仅在学习科研的过程中给予我很多教育和指导，具体地林老师在日常科研工作中严谨又不失亲切，在科研项目、研究生论文选题和研究等方面给予了很多十分重要的意见和建议。更为重要的是，林老师严谨不苟的治学态度、平易宽厚的生活作风等都将对我的一生受用无穷。在此，对谢长生老师、李国宽老师、胡迪青老师、王海卫老师、姚杰老师、黄浩老师等课题组全体老师表示感谢，感谢你们在科研工作和日常事务上忘我的付出和热情的帮助。

再次，我要对研究生期间的同窗挚友表示由衷的感谢。感谢实验室同窗李升亮、张洪泉、曹伟、陈斯，在科研的过程中，是你们一直陪伴着我、鼓励我以及帮助我，让我在三年的科研学习过程不仅收获了知识，也获得了快乐和友谊；感谢室友汪志明、王同雷、程佳珉，是你们在我的日常生活中给予了我无限的欢笑；同时感谢硕1202班和光电F103的所有同学，感谢三年的学习生活中有你们，使我们的硕士研究生生涯中充满色彩。

最后，在此对所有给予我关切帮助的人表示由衷的感谢！

参考文献

- [1] Ammon G J, Calabria J A, Thomas D T. A high-speed, large-capacity, 'jukebox' optical disk system. IEEE Computer, 1985, 18(7): 36-45
- [2] 周之恒. 用于容灾系统的纠删码技术研究: [硕士学位论文]. 上海: 上海交通大学图书馆, 2010
- [3] 李明强. 磁盘阵列的纠删码技术研究: [博士学位论文]. 北京: 清华大学图书馆, 2011
- [4] 姜泽雄. 纠删码在网络存储系统中的实现与可靠性仿真: [硕士学位论文]. 成都: 电子科技大学图书馆, 2010
- [5] Meggitt J E. Error correcting codes and their implementation for data transmission systems. IEEE Transactions on Information Theory, 1961, 7(4): 35-60
- [6] Blaum M, Brady J, Bruck J, et al. EVENODD: An efficient scheme for tolerating double disk failures in RAID architectures. IEEE Transactions on Computers, 1995, 44(2): 192-202
- [7] Xiang L, Xu Y, Lui J, et al. Optimal recovery of single disk failure in RDP code storage systems. In: Proceedings of the ACM SIGMETRICS Performance Evaluation Review. San Diego: ACM Press, 2010: 119-130
- [8] 兰玉龙. 容双盘错 RDP 算法的设计与实现: [硕士学位论文]. 武汉: 华中科技大学图书馆, 2007
- [9] Huang C, Xu L. STAR: An efficient coding scheme for correcting triple storage node failures. IEEE Transactions on Computers, 2008, 57(7): 889-901
- [10] Xu L, Bruck J. X-code: MDS array codes with optimal encoding. IEEE Transactions on Information Theory, 1999, 45(1): 272-276
- [11] Reed I S, Solomon G. Polynomial codes over certain finite fields. Journal of the Society for Industrial & Applied Mathematics, 1960, 8(2): 300-304
- [12] Plank J S. A tutorial on Reed-Solomon coding for fault-tolerance in RAID-like systems. Software: Practice and Experience, 1997, 27(9): 995-1012
- [13] Plank J S, Ding Y. Note: Correction to the 1997 tutorial on Reed-Solomon coding.

- Software: Practice and Experience, 2005, 35(2): 189-194
- [14] Blomer J, Kalfane M, Karp R, et al. An XOR-based erasure-resilient coding scheme. International Computer Science Institute, UC Berkeley, Technical Report TR-95-048, 1999: 1-19
- [15] Plank J S, Xu L. Optimizing Cauchy Reed-Solomon codes for fault-tolerant network storage applications. In: Proceedings of the 5th IEEE Symposium on Network Computing and Applications. Cambridge, Massachusetts: IEEE Press, 2006: 173-180
- [16] Feng G L, Deng R H, Bao F, et al. New efficient MDS array codes for RAID Part I: Reed-Solomon-like codes for tolerating three disk failures. IEEE Transactions on Computers, 2005, 54(9): 1071-1080
- [17] Feng G L, Deng R H, Bao F, et al. New efficient MDS array codes for RAID Part II: Rabin-like codes for tolerating multiple (≥ 4) disk failures. IEEE Transactions on Computers, 2005, 54(12): 1473-1483
- [18] Hafner J L. HoVer erasure codes for disk arrays. In: Proceedings of the 2006 International Conference on Dependable Systems and Networks. Philadelphia: IEEE Press, 2006: 217-226
- [19] Gallager R G. Low-density parity-check codes. IEEE Transactions on Information Theory, 1962, 8(1): 21-28
- [20] Blaum M. A family of MDS array codes with minimal number of encoding operations. In: Proceedings of the 2006 IEEE International Symposium on Information Theory. Seattle, Washington: IEEE Press, 2006: 2784-2788
- [21] Chen P M, Lee E K, Gibson G A, et al. RAID: High-performance, reliable secondary storage. ACM Computing Surveys, 1994, 26(2): 145-185
- [22] 朱卫卫. 基于重码和二进制矩阵的 RAID 编码算法研究: [硕士学位论文]. 长沙: 湖南大学图书馆, 2011
- [23] Huang C, Simitci H, Xu Y, et al. Erasure Coding in Windows Azure Storage. In: Proceedings of the 2012 USENIX Annual Technical Conference. Boston: USENIX Association, 2012: 15-26
- [24] Plank J S. The raid-6 liberton code. International Journal of High Performance

- Computing Applications, 2008: 97-110
- [25] Plank J S. The raid-6 liberation code. International Journal of High Performance Computing Applications, 2009: 242-251
- [26] 毕文娟. 一种基于纠删码的分布式备份技术的研究: [硕士学位论文]. 武汉: 华中科技大学图书馆, 2013
- [27] Anderson E, Li X, Merchant A, et al. Efficient eventual consistency in Pahoehoe, an erasure-coded key-blob archive. In: Proceedings of the 2010 International Conference on Dependable Systems and Networks. Chicago: IEEE Press, 2010: 181-190
- [28] Schwarz T J E, Xin Q, Miller E L, et al. Disk scrubbing in large archival storage systems. In: Proceedings of the 12th International Workshop on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems. Vollandam: IEEE Press, 2004: 409-418
- [29] Hafner J L, Deenadhayan V, Rao K K, et al. Matrix Methods for Lost Data Reconstruction in Erasure Codes. In: Proceedings of the 4th USENIX Conference on File and Storage Technologies. San Jose: USENIX Association, 2005: 15-30
- [30] Sathiamoorthy M, Asteris M, Papailiopoulos D, et al. Xoring elephants: Novel erasure codes for big data. In: Proceedings of the VLDB Endowment. Riva del Garda, Italy: DISI, 2013: 325-336
- [31] 罗象宏, 舒继武. 存储系统中的纠删码研究综述. 计算机研究与发展, 2012, 49(1): 1-11
- [32] 程振东, 栾钟治, 孟由等. 云文件系统中纠删码技术的研究与实现. 计算机科学与探索, 2013, 7(4): 315-325
- [33] Plank J S, Huang C. Tutorial: Erasure coding for storage applications. In: Proceedings of the 11th USENIX Conference on File and Storage Technologies. San Jose: USENIX Association, 2013: 10-24
- [34] 鄢喜爱. 基于 RS 编码的存储容错分析. 网络安全技术与应用, 2014(9): 62-63
- [35] 徐朝军. RS 码译码算法及其实现的研究: [博士学位论文]. 西安: 西安电子科技大学图书馆, 2006

- [36] 彭建明. 基于 Reed-Solomon 编码的数据容错算法的设计与实现: [硕士学位论文]. 长沙: 国防科学技术大学图书馆, 2006
- [37] Plank J S. Fast Galois field arithmetic library in C/C++. University of Tennessee, Tech. Rep. UT-CS-07-593, 2007: 1-9
- [38] Schuman C D, Plank J S. A performance comparison of open-source erasure coding libraries for storage applications. In: Proceedings of the 7th USENIX Conference on File and Storage Technologies. San Jose: USENIX Association, 2009: 253-265
- [39] Corbett P, English B, Goel A, et al. Row-diagonal parity for double disk failure correction. In: Proceedings of the 3th USENIX Conference on File and Storage Technologies. San Jose: USENIX Association, 2004: 1-14
- [40] Hafner J L. WEAVER Codes: Highly Fault Tolerant Erasure Codes for Storage Systems. In: Proceedings of the 4th USENIX Conference on File and Storage Technologies. San Jose: USENIX Association, 2005: 16-16
- [41] Bairavasundaram L N, Goodson G R, Pasupathy S, et al. An analysis of latent sector errors in disk drives. In: Proceedings of the ACM SIGMETRICS Performance Evaluation Review. San Diego: ACM Press, 2007: 289-300
- [42] Plank J S, Blaum M, Hafner J L. SD codes: erasure codes designed for how storage systems really fail. In: Proceedings of the 11th USENIX Conference on File and Storage Technologies. San Jose: USENIX Association, 2013: 95-104
- [43] Plank J S, Blaum M. Sector-disk (SD) erasure codes for mixed failure modes in RAID systems. ACM Transactions on Storage, 2014, 10(1): 4-15
- [44] Tian L, Feng D, Jiang H, et al. PRO: A Popularity-based Multi-threaded Reconstruction Optimization for RAID-Structured Storage Systems. In: Proceedings of the 5th USENIX Conference on File and Storage Technologies. San Jose: USENIX Association, 2007: 301-314
- [45] Plank J S, Simmerman S, Schuman C D. Jerasure: A library in C/C++ facilitating erasure coding for storage applications. Technical Report CS-08-627, University of Tennessee, 2008: 1-39
- [46] Schroeder B, Gibson G A. Understanding disk failure rates: What does an MTTF of 1, 000, 000 hours mean to you?. ACM Transactions on Storage, 2007, 3(3): 1-31